

西安电子科技大学

硕士学位论文



基于改进蚁群算法的超大规模集
成电路布线研究

作者姓名 郭皓天

学校导师姓名、职称 陈玉峰 副教授

企业导师姓名、职称 吴启俊 高工

申请学位类别 工程硕士

学校代码 10701
分 类 号 TN47

学 号 19041212087
密 级 公开

西安电子科技大学

硕士学位论文

基于改进蚁群算法的超大规模集成电路布线研究

作者姓名：郭皓天

领 域：控制工程

学位类别：工程硕士

学校导师姓名、职称：陈玉峰 副教授

企业导师姓名、职称：吴启俊 高工

学 院：机电工程学院

提交日期：2022 年 3 月

VLSI routing based on improved ant colony algorithm

A thesis submitted to
XIDIAN UNIVERSITY
in partial fulfillment of the requirements
for the degree of Master
in Engineering

By

Guo Hao Tian

Supervisor: Chen Yu Feng Title: Associate Professor

Supervisor: Wu Qi Jun Title: Senior Engineer

March 2022

摘要

集成电路在现代生活中随处可见，它从设计到生产制造的每一个环节都涉及多种学科和技术。随着集成电路集成度不断提高、特征尺寸不断缩小并接近物理极限，互连线带来的延时和功耗成为影响电路性能的瓶颈。在此背景下，本文通过改进的蚁群算法解决布线问题，主要内容如下：

（1）本文通过旅行商问题（Traveling Salesman Problems, TSP），介绍了蚁群算法的原理和数学模型。最大最小蚁群系统（Max-Min Ant System, MMAS）和改进的概率选择公式，避免蚁群算法在局部的停滞，提升搜索速度。使用无网格的集成电路布线模型，相较于有网格模型，无网格模型的访问点阵规模更小，需要的存储空间更少，搜索速度更快。

（2）对于蚁群算法搜索到的路径之间容易相互干扰、强化新路径速度慢这两个问题，引入了河流形成动力（River Formation Dynamics, RFD）算法。RFD 算法根据两点间的梯度决定选择概率，初始点和目标点的高度差相同，所以距离越短的路径梯度就越大，该路径也更有可能被选择。

（3）双端布线是最简单的布线形式，还有以双端布线为基础的多端布线和多组布线。多端布线不是多个双端布线的简单组合，它可以转化为最小矩形斯坦树问题（Minimum Rectangular Steiner Tree, MRST）。而多组布线问题要考虑每个线网优先级，在布线过程中可以动态调整线网优先级实现所有线网布通。通过将蚁群算法、RFD 算法和混合算法分别应用到双端布线、多端布线和多组布线问题中，实验结果证明了混合算法的寻优能力更强。

（4）在多层布线中，层与层之间通过通孔连接，需要尽可能减少通孔的使用。本文使用的多层布线算法首先允许单层交叉走线，存在交叉就会产生通孔。通过将交叉的边分配到最少的层数中达到减少通孔的目的。

关键词：超大规模集成电路，蚁群算法，无网格布线，河流形成动力

ABSTRACT

Integrated circuits can be seen everywhere in modern life, and every process from design to manufacturing involves a variety of disciplines and technologies. With the continuous improvement of integrity, the feature size is shrinking and approaching the physical limit. Therefore, the delay and power consumption caused by interconnect lines become the bottleneck affecting circuit performance. In this context, this thesis solves the routing problem through the improved ant colony algorithm. The main contents are as follows:

(1) This thesis introduces the principle and mathematical model of the ant colony algorithm through traveling salesman problems (TSP). The Max-Min Ant System(MMAS) and optimized probability formula can avoid the local stagnation of the ant colony algorithm and improve the search speed according to the relative relationship between the next point and the target point. A gridless model is used in the routing model of integrated circuits. Compared with the grid model, the access lattice of the gridless model has a smaller scale, less storage space, and faster search speed.

(2) The river formation dynamics (RFD) algorithm is introduced in order to address the two problems that the paths searched by the ant colony algorithm are easy to interfere with each other and the speed of strengthening the new path is slow. The RFD algorithm determines the selection probability according to the gradient between the two points. The initial point and the target point have the same altitude difference. So the shorter the distance is, the greater the gradient becomes, and the more possibly the path is selected.

(3) Two-terminal net routing is the simplest form of routing. Besides there are multi-terminal net routing and multi-group net routing based on two-terminal net routing. Multi-terminal net routing is not a simple combination of multiple two-terminal net routing. It can be transformed into the problem of the Minimum Rectangular Steiner Tree (MRST). However, for the problem of multiple-group net routing, the priority of each net should be considered, and the priority of the net can be dynamically adjusted during the routing process to connect all nets. By applying the ant colony algorithm, the RFD algorithm, and their hybrid algorithm to the problems of two-terminal net routing, multi-terminal net routing, and multi-group net routing, respectively, the experimental results show that the hybrid algorithm has a stronger

optimization ability.

(4) The layers are connected by vias, the number of which needs to be gotten as lower as possible in the multi-layer routing. The multi-layer routing algorithm used in this thesis first allows single-layer cross routing, and the existence of cross may produce vias. The number of vias is reduced by allocating the crossed edges to the minimum number of layers.

Keywords: VLSI, ant colony algorithm, gridless routing, RFD

插图索引

1.1	2011-2019 中国芯片进出口价值及贸易逆差	2
2.1	VLSI 设计流程	10
2.2	物理设计的不同阶段	11
2.3	分层设计方法	12
2.4	BBL 布图模式	12
2.5	布局草图的过度	14
2.6	两种布线方式	15
2.7	通道布线问题	15
2.8	通道问题分解	16
3.1	蚁群觅食过程	17
3.2	蚁群算法解决 TSP 的流程	20
3.3	不同架构布线模型	23
3.4	不同架构布线结果	23
3.5	器件在不同大小网格的影响	24
3.6	垂直轨线的构造过程	26
3.7	访问点阵的形成	27
3.8	拐点示意图	28
3.9	累计概率	28
3.10	布线图	29
3.11	布线结果图	29
4.1	RFD 算法原理示意图	31
4.2	RFD 算法执行流程	32
4.3	混合算法解决双端布线问题	36
4.4	起点不同的多端布线	37
4.5	布线顺序的影响	37
4.6	导线的线宽和线距	38
4.7	双端布线	39
4.8	三种算法的收敛情况	39
4.9	双端布线的迭代过程	40
4.10	双端布线最优值和平均值的对比	40

4.11 多端布线	41
4.12 多端布线迭代过程	42
4.13 多端布线最优值和平均值的对比	42
4.14 多组布线	43
4.15 多组布线迭代过程	43
4.16 多组布线最优值和平均值的对比	44
4.17 约束通孔最小化	45
4.18 无约束通孔最小化	45
4.19 多层布线初始结果	45
4.20 顶点交叉图	47
4.21 多层布线最终结果	47

表格索引

3.1	蚁群算法参数	29
4.1	双端布线实验数据	40
4.2	多端布线实验数据	41
4.3	多组布线实验数据	43
4.4	矩阵 M	46
4.5	多层布线实验结果	49

符号对照表

符号	符号名称
d_{ij}	点 i 和点 j 的距离
τ	信息素浓度
α	信息素启发因子
β	启发函数重要因子
$allowed_k$	未访问节点集合
ρ	信息素挥发因子
Q	蚂蚁携带信息素之和
L_k	蚂蚁 k 搜索到的路径长度
Nc_{max}	最大迭代次数
p^{ib}	迭代最优蚂蚁
p^{gb}	全局最优蚂蚁
$G(O, s, t)$	路径图
$G(V, E)$	访问点阵
f_k	线网 k 的成本函数
α_l	路径长度权重
β_v	拐点代价函数权重
ω	概率选择常数
ϵ	侵蚀参数

缩略语对照表

缩略语	英文全称	中文对照
RFD	River Formation Dynamics	河流形成动力
MRST	Minimum Rectangular Steiner Tree	最小矩形斯坦纳树
PCB	Printed Circuit Board	印制电路板
CMOS	Complementary Metal Oxide Semiconductor	互补金属氧化物半导体
SSI	Small Scale Integration	小规模集成电路
MSI	Medium Scale Integration	中规模集成电路
LSI	Large Scale Integration	大规模集成电路
VLSI	Very Large Scale Integration	超大规模集成电路
ULSI	Ultra Large Scale Integration	特大规模集成电路
EDA	Electronic Design Automation	电子设计自动化
LVS	Layout Versus Schematic	布局与原理图
SoC	System-on-Chip	片上系统
PLD	Programmable Logic Device	可编程逻辑器件
ASIC	Application Specific Integrated Circuit	专用集成电路
SI	Swarm Intelligence	群体智能
DFS	Depth First Search	深度优先搜索
BFS	Breadth First Search	广度优先搜索
FPGA	Field Programmable Gate Array	现场可编程门阵列
BBL	Block Building Layout	积木块布图
TSP	Traveling Salesman Problems	旅行商问题
MMAS	Max-Min Ant System	最大最小蚁群系统
SP	Shortest Path	最短路径
MCP	Minimun-Cost Path	最小费用路径
ACO	Ant Colony Optimization	蚁群优化

TSV	Through Silicon Via	硅通孔
CVM	Constrained Via Minimization	通孔约束最小化
UVM	Unconstrained Via Minimization	无约束通孔最小化

目 录

第一章 绪论.....	1
1.1 课题背景和研究意义.....	1
1.2 集成电路的发展.....	3
1.3 EDA 技术的发展	4
1.4 国内外研究现状.....	5
1.4.1 蚁群算法研究现状.....	5
1.4.2 集成电路布线研究现状.....	6
1.5 论文内容安排.....	6
第二章 VLSI 的设计流程.....	9
2.1 设计流程介绍.....	9
2.2 物理设计.....	10
2.2.1 电路划分	10
2.2.2 布局规划.....	11
2.2.3 布线.....	13
2.2.4 压缩.....	13
2.3 全局布线.....	14
2.4 详细布线.....	14
2.5 本章小结.....	16
第三章 基于蚁群算法的布线研究	17
3.1 蚁群算法的介绍.....	17
3.1.1 蚁群算法的原理.....	17
3.1.2 蚁群算法的实现过程.....	18
3.1.3 蚁群算法的优化.....	20
3.2 布线模型的路径图.....	22
3.2.1 集成电路布线模型.....	22
3.2.2 路径图模型.....	24
3.2.3 路径图的构造.....	26
3.3 蚁群算法在布线问题的应用	27
3.3.1 优化目标.....	27
3.3.2 选择机制.....	28
3.3.3 蚁群算法用于布线问题的演示.....	28
3.4 本章小结.....	30
第四章 改进蚁群算法应用于 VLSI 布线	31
4.1 RFD 算法的介绍.....	31

4.1.1 RFD 算法的原理	31
4.1.2 RFD 算法的实现过程	31
4.2 基于 ACO-RFD 的混合算法	34
4.2.1 蚁群算法和 RFD 算法的比较	34
4.2.2 ACO-RFD 的混合算法	35
4.3 基于 ACO-RFD 的混合算法的应用	36
4.3.1 多端、多组布线问题	36
4.3.2 加入线宽和线距约束	37
4.3.3 布线问题的解决	38
4.4 多层布线	44
4.5 本章小结	49
第五章 总结与展望	51
5.1 总结	51
5.2 展望	51
参考文献	53

第一章 绪论

1.1 课题背景和研究意义

集成电路的发展经历了电子管、晶体管和集成电路三个阶段^[1]。在集成电路发明之前，人们就能将电阻、电容、二极管和晶体管等单个电子元器件组装到印制电路板（Printed Circuit Board, PCB），并通过外部接线组成所需要的电路。集成电路的出现不仅使电子设备轻小化，还使其具有功能复杂多样、高速、节能等特点，而且由于其焊点较少，元器件具有更高的品质和更低的功耗。目前 CMOS（Complementary Metal Oxide Semiconductor）工艺量产技术已经进入 3nm 工艺节点，随着特征尺寸的缩小，线宽、互连线的功耗、延时和光刻分辨率都已经接近极限；量子力学的作用开始凸显，这些问题标志着摩尔定律的失效，三维结构的集成电路是后摩尔时代的重要发展方向^[2]。在此背景下 IEEE 重启了新的国际会议来研究未来集成电路的发展问题^[3]。

集成电路首先被应用于航空航天、卫星通信和辐射防护领域的国防系统^[4]。对于小规模集成电路（Small Scale Integration, SSI），它的工艺简单，制造成本低，在那个时期几乎每个系统公司都有自己的晶圆厂。随后集成电路很快就进行了商业应用，尤其在计算机中得到了广泛应用：从大型机到小型机，然后是工作站和个人电脑。同时在摩尔定律的指导下，特征尺寸的缩小把集成电路推广到了中规模集成电路（Medium Scale Integration, MSI）、大规模集成电路（Large Scale Integration, LSI）、超大规模集成电路（Very Large Scale Integration, VLSI）和特大规模集成电路（Ultra Large Scale Integration, ULSI），由 CMOS 技术制造的超大规模集成电路在生产生活中随处可见。

集成电路是现代信息产业和信息社会的基石，大数据与云计算、人工智能、通信技术、生物技术、能源和先进制造等领域都对集成电路技术存在高度依赖。集成电路也是国家重大战略需求的一部分，政府规划的“新基建”中许多产业的发展都离不开集成电路。在“十四五”计划中^[5]，我国集成电路产业将迈向高质量发展阶段，物联网、人工智能和自动驾驶等领域的兴起给集成电路产业的发展带来很大机会。在美国制裁和新冠疫情的影响下，全球集成电路产业分工体系遭到严重破坏，导致了芯片产量降低、供应短缺的问题，加上国家政策的扶持，国内各大厂商也有很大意愿自研芯片，开始加大对集成电路领域的投资；2020 年教育部把“集成电路科学与工程”专业设为一级学科，国内高校也更加注重对半导体行业人才的培养。

由于早期我国对集成电路领域的投资太少，导致历史欠账太多，短时间内难以弥补，2020 年我国集成电路产业需要技术人员 72 万人，而集成电路技术人才存量只有

40 万人，缺口达到 32 万人。此外，在集成电路人才培养方面也存在欠缺，“十三五”期间培养的相关技术人员仅有 7.5 万人，完全不能满足产业发展。我国对集成电路的需求很大，2011-2019 年中国芯片进出口价值及贸易逆差如图 1.1 所示，基本上每年进口额都在增加，出口额增长缓慢，甚至会出现下降的情况。中国在 2020 年进口集成电路 5435 亿块，价值 3800 亿美元，而同年进口石油只有 1763 亿美元。由于美国政府对中国企业的相关制裁限制，许多中国公司在制裁生效前订购大量芯片，2020 年中国芯片进口激增了约 14%，连续 3 年进口集成电路超过 3000 亿美元。集成电路行业的回报十分丰厚，2020 年美国在集成电路行业的收入达到了 2260 亿美元，全球市场占有率达到了 48%，毛利润率达到了 62%。台积电（TSMC）作为一个集成电路制造商，它在 2020 年的净收入达到了 455.1 亿美元，毛利润率为 53.1%，净利润率为 42.3%，利润十分惊人。2022 年，台积电成熟制程将调涨 10% 至 20%，营收能力将进一步增加。从以上情况可以看到，集成电路行业利润巨大，对集成电路产业的投资十分有必要，不仅能掌握核心竞争力，摆脱在相关设备、材料受制于人的情况，还能带动其他相关产业的发展，增强我国集成电路产业在国际上的竞争力。

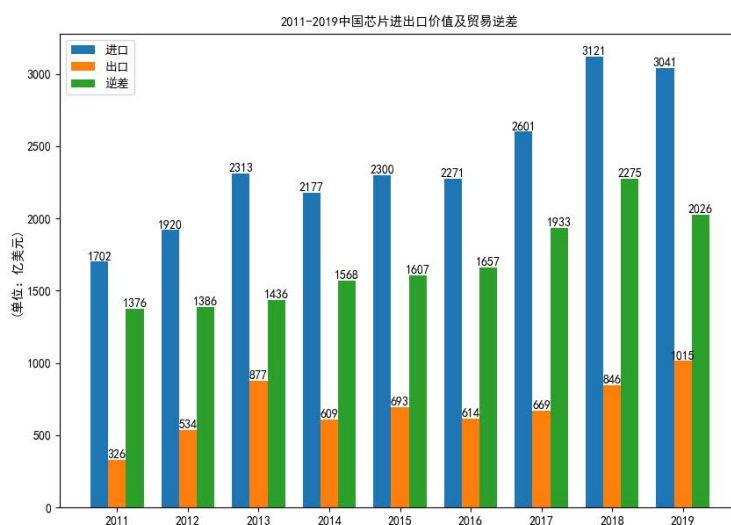


图 1.1 2011-2019 中国芯片进出口价值及贸易逆差

直到上世纪七十年代末，集成电路的物理设计（Physical Design）工作仍是由人工来完成的，但随着集成电路的规模不断扩大，设计电路仅靠人力是很难实现，必须依靠电子设计自动化（Electronic Design Automation, EDA）工具来完成。随着集成电路的飞速发展，互连线的延时和功耗问题成为影响电路性能的瓶颈，从而集成电路的布线问题成为提高电路性能的关键所在。目前市场占有率高的 EDA 工具基本都是由西方国家的企业开发的，每年的授权费用十分高昂，小微企业根本无法承担。2020 年上半年，美国对华为的制裁更为严格，华为设计芯片的 EDA 软件授权在 2021 年就将停止，台积电等制造商也不能为华为生产芯片，这使得华为在高端手机市场受到重

创，也迫使华为不得不研发自己的 EDA 工具。这再次证明我国在集成电路制造设备、先进材料和工艺以及 EDA 工具的自主性不足，受国外制约极其严重^[6]。我国有超过万亿的集成电路市场需求，所以对集成电路产业需要的设备、材料和 EDA 软件都要加大投入，掌握核心技术，增强自身竞争力。

1.2 集成电路的发展

集成电路是通过氧化、光刻、刻蚀、扩散、蒸发和封装等关键工艺，把一整套电路中所需要的二极管、晶体管和电容、电阻、电感等元件制造在一块基片上，用一定的方法连接并封装，使之成为具有一定系统功能的微结构。1904 年英国物理学家弗莱明发明了二级检波管，随后 1907 年美国发明家弗雷斯特发明了真空三极管，到了 1946 年世界上第一台电子计算机被制造出来，这是集成电路的起源阶段。贝尔实验室在 1947 年演示了点接触式晶体管，1954 年开发了第一台晶体管计算机，这是集成电路的初型阶段。1958 年杰克·基尔比发明了集成电路，1965 年仙童公司的摩尔提出了著名的摩尔定律，集成电路产业就沿着摩尔定律指引的方向飞速发展^[1]。近些年，集成电路特征尺寸已经发展到接近物理极限，摩尔定律几乎已经失效，对于后续集成电路的发展，已经有不同方向的研究正在进行当中。

以往的集成电路仅在二维平面上设计，发展到现在这个阶段，在平面上缩小器件尺寸的难度急剧增加，有公司研究集成电路的纵向设计，通过芯片的堆叠在相同面积下放置更多器件。三维集成电路不仅在器件密度上有所提高，还使得信息传输距离大幅缩减，时钟同步难度降低、功耗降低约 40%；拥有更有效的同步化并行处理结构，极大丰富了 IO 资源；三维集成结构的处理器，其逻辑器件与存储器件间距更小，传输线路更短，两者之间的传输延时和功耗都会降低，使处理器芯片性能得到提升。例如，当存储器件和逻辑芯片在一个平面时，它们要不断通过 IO 总线读取、存入数据，大量的时间和功率消耗在数据传输过程中，这样会产生更多的延时和能耗。如果向三维方向发展，将存储器件放在上方，逻辑芯片放在下方，就可以最大可能减少线路长度，从而加快芯片处理速度、减少芯片功耗。2021 年，IBM 实现了 2nm 工艺技术，该技术就是通过芯片在三维上的堆叠使得在单位面积上器件数量更多，它是综合性能的评估而不是物理尺度达到 2nm。

以硅为主体的半导体晶体管很难支撑集成电路产业继续发展，在特征尺寸不变的情况下使用电子迁移率更高的材料也能起到提升电路性能的作用。如果使用新型碳材料作为制造芯片的材料，其电子迁移率几乎是硅的 10 倍以上，也就是说目前 10nm 的硅器件，用 100nm 的碳纳米管就可以实现相同性能，这样就不用特别关注特征尺寸的问题，也就不需要特别先进的光刻机制造芯片。

此外还有一种发展方向为异质集成。这种理念是不再用最先进的工艺制造所有

器件，芯片的存储、总线和互联没有必要使用最先进的工艺。通过将功能、工艺不同的芯片集成在一块硅基片上，可以达到降低成本、提升性能或完善功能的作用。例如，在一些性能要求不高的器件上可以不使用最先进的制程工艺来降低成本，而对处理器这类要求很高的器件可以采用最先进的工艺并且使用砷化镓、氮化镓等材料来进一步提升性能。

1.3 EDA 技术的发展

集成电路的物理设计最初是由手工完成的，计算机是绘制布局的工具，随着集成电路中组件数量的增加，人工设计不再能满足物理设计的要求，出现了对物理设计自动化工具的强烈需求。第一批 EDA 工具与布局设计有关，例如布局编辑器和设计规则检查器，随着芯片的组件密度不断增大，需要新的 EDA 工具用作更高的抽象设计级别。如今 EDA 工具的分类如下^[7]：

合成工具：使用一个较高抽象层次的电路描述作为输入，并提供另一个较低抽象层次的电路描述作为输出的工具。

验证工具：它的作用是验证合成过程中是否存在错误，常见的工具如模拟工具、正式验证工具和布局与原理图（Layout Versus Schematic, LVS）。

优化工具：这些工具使用一个抽象级别的电路作为输入，并提供相同抽象级别的另一个描述作为输出，但有针对性功率、时序或面积的优化。

评估工具：这种类型的工具越来越重要，因为它们提供的评估精度足以帮助任何抽象层次的决策，这也意味着需要在每个抽象层次上使用不同的评估工具。

随着电子技术、微电子技术和计算机技术的发展，EDA 在近几十年有了很大的发展，它涵盖了电子设计、仿真、验证和制造过程的所有技术，如：系统设计与仿真、电路设计与仿真、印刷电路板设计与验证、集成电路版图设计、验证与测试、数字逻辑电路设计、模拟电路设计、混合信号设计、嵌入式系统设计、软硬件系统设计、片上系统（System-on-Chip, SoC）设计、可编程逻辑器件（Programmable Logic Device, PLD）和可编程片上系统设计、专用集成电路（Application Specific Integrated Circuit, ASIC）和专用标准产品（Application Specific Standard Products, ASSP）设计技术^[8]。EDA 应用领域包括机械、电气、通信、航空航天、化工、矿产、生物、医学、军事等领域。例如，在飞机的制造、设计、性能测试和性能分析，直到飞行模拟，EDA 技术都会参与其中。EDA 的发展可以分为四个阶段^[9]：

计算机辅助设计阶段：使用计算机辅助进行集成电路版图设计、布局布线；

计算机辅助工程阶段：主要对设计电路的功能检测问题进行处理；

电子系统设计自动化阶段：EDA 技术逐渐以高级语言描述、系统仿真和综合优化为特征；

现阶段 EDA 技术广泛地运用到设计、性能测试、特性分析、产品模拟等阶段。

EDA 技术的发展需要长期、稳定、高强度的研发投入，以新思科技（Synopsys）和楷登电子（Cadence）为代表的美国 EDA 企业，在部分年份出现亏损的情况下，研发占比仍能高于 30%。而中国 EDA 厂商虽然数量很多，大多仍是点工具开发，所对应的细分领域和企业所处地理位置都相对比较分散^[10]。

1.4 国内外研究现状

1.4.1 蚁群算法研究现状

蚁群算法作为一种很流行的元启发式算法起源于 20 世纪 90 年代初，由意大利学者 Dorigo M 等人在 1991 年首次提出^[11]。蚁群算法最先在 TSP 上得到应用，实验结果表明它可以很快收敛到最优解。此后，从事计算机科学、路径规划、管理科学、控制科学、生物科学等领域的研究人员成功地将蚁群算法扩展和应用到 DNA 测序^[12]、软件测试^[13]、调度问题^[14]、二次分配问题^[15]、蛋白质配体对接^[16]、装配线平衡^[17]、路径规划^[18-19]、系统辨识^[20-21] 和图像处理^[22-23] 等领域。林树峰等人^[24] 用蚁群算法对水下机械臂进行路径规划，在时间和能量两个优化目标上均能节省 50% 以上，大大提高了机械臂的工作效率。田云娜等人^[25] 将基于时间窗的蚁群算法应用于全局调度问题的求解，减少了蚁群算法的随机性，实验结果表明改进算法既减少了机器的等待时间，又提升了算法的计算效率。焦留成等人^[26] 将蚁群算法应用于求解连续空间约束优化问题，加入了惩罚函数，在各项数据上都优于改进前的算法。Mavrovouniotis 和 Yang^[27] 将蚁群算法应用到动态车辆路径问题，在实际用例和仿真结果上都取得了较好的结果。Ramkumar 等人^[28] 将基于种群的混合蚂蚁算法用于布局设计的二次分配问题，与禁忌搜索、模拟退火和遗传算法的结果相比较，实验结果表明改进后的算法效果更好。

对于蚁群算法的信息素机制，也有一些改进方法。毛嘉琪^[29] 提出首先使用改进的 A* 算法快速得到一条较优路径来优化信息素初值，再按照蚁群算法进行搜索，仿真实验证明了该算法的有效性和可行性。任红格等人^[30] 同时基于算法的时间信息和地图空间信息来更新信息素，引入最小挥发系数，实验表明改进后的蚁群算法寻优能力更强、收敛速度更快。Ismkhan^[31] 提出一种新的信息素更新方法，利用动态信息进行增强信息素更新，而且当蚁群搜索接近停滞时初始化信息素重新进行搜索。Luo 等人^[32] 提出一种改进的蚁群算法，构建了非均匀分配的初始信息素矩阵，避免初期蚁群的盲目搜索，引入最优解和最差解来改进全局信息素更新方法，改进后的无效蚂蚁数量不到其他算法的三分之一。Yu 等人^[33] 提出一种新的更新信息素的蚂蚁权重策略，并应用到车辆路径规划问题。

对于蚁群算法容易陷入局部最优和收敛速度较慢的问题, 张天瑞等人^[34]将机器人的转角启发函数与概率选择公式相结合, 使路径选择更具有指向性, 提高了算法搜索性能; 提出信息素挥发引子的自适应调整方法, 扩大算法搜索范围, 提高了算法收敛速度。Zhang 等人^[35]提出了路径优化和个体变异的改进方法, 前者减少了蚂蚁寻路的时间, 后者使蚂蚁拥有不同的搜索策略, 提高了蚁群算法的收敛速度。Deng 等人^[36]提出一种基于多种群策略、协同进化、信息素更新和扩散机制的改进蚁群算法, 在大规模的优化问题有更好的表现, 在 TSP 和登机口分配问题的仿真实验取得了较好的效果。Escario 等人^[37]将蚂蚁根据任务分为巡逻蚁和觅食蚁, 通过一个调节策略控制每种蚂蚁的数量, 在 TSP 的仿真中表明算法有较好的优化性能。

1.4.2 集成电路布线研究现状

董晨^[38]提出了基于惯性权值自适应调整的粒子群优化算法, 改进了粒子群算法的收敛速度和寻优能力, 使得算法更适用于 VLSI 全局布线问题。Jose 等人^[39]通过蚁群算法优化 VLSI 电路的划分, 确定可测试性设计的最佳位置进行电路布线故障检测。Peyer 等人^[40]推广了 Dijkstra 算法, 在有向图中搜索最短路径使之可以应用到布线问题, 算法可以分别应用在子图划分和详细布线当中, 通过面向目标的技术大大减少了运行时间。Yang 等人^[41]使用基于整数规划的全局布线模型, 实验表明该模型能够有效地同时优化多个全局布线目标。Deng 等人^[42]将广度优先搜索 (Breadth First Search, BFS) 和深度优先搜索 (Depth First Search, DFS) 相结合, 寻找 VLSI 布线中的所有路径, BFS 用于计算当前位置到起始位置的最短距离, DFS 用于遍历从终点到起点回溯过程中的所有最短路径。Manna 等人^[43]使用差分进化的优化技术来寻找布线问题的最短路径, 实验结果表明该算法在小维度和大维度的布线问题中都表现良好。Ayob 等人^[44]使用萤火虫算法来选择时间延迟最小的路径, 实例仿真结果表明该方法具有进一步推广的潜力。Nath 等人^[45]提出了离散粒子群算法的两个变体: 线性递减惯性权重和自调整加速度常数, 结果表明该算法有更好的收敛速度、稳定性和更低的时间复杂度。

1.5 论文内容安排

第一章首先介绍了课题背景和研究意义, 讨论集成电路的发展状况和未来研究趋势以及 EDA 技术的发展情况。最后介绍了蚁群算法和集成电路布线研究的国内外研究现状。

第二章首先介绍了 VLSI 的设计流程, 着重介绍了物理设计部分的电路划分、布局规划、布线和压缩的作用, 最后介绍了布线问题中的全局布线和详细布线问题。

第三章首先介绍了蚁群算法的原理, 通过 TSP 的实现过程给出蚁群算法的数学

模型和两个可优化部分。其次介绍集成电路布线模型的建立和路径图的构造过程。最后介绍了布线问题中的优化目标和选择机制，并对有网格模型和无网格模型的布线结果进行对比。

第四章首先介绍了 RFD 算法的原理和实现过程。其次分析了蚁群算法存在的不足并与 RFD 算法进行比较，形成了蚁群算法和 RFD 算法的混合算法。随后介绍了多端、多组布线问题并在双端、多端和多组布线问题上进行仿真实验。最后介绍了多层布线的具体方法。

第五章对本文的研究工作进行总结，指出当前研究的不足之处和对后续研究工作的期望。

第二章 VLSI 的设计流程

2.1 设计流程介绍

集成电路从设计到生产制造, 每一个环节都涉及了许多学科和技术, 它是分阶段开发的。VLSI 的设计流程可以分为系统描述、功能设计、逻辑设计、电路设计、物理设计、设计验证、电路制造和测试封装^[46]。

(1) 系统描述。根据设计电路的目标和需求, 确定整个系统所需要的具体功能、性能指标、物理尺寸、制造工艺、设计周期和成本^[47]。

(2) 功能设计。考虑系统要实现的不同功能, 将整个系统分为若干子模块, 按照需求将功能分配到 Soc 或者电路板上, 给出子模块之间的连接关系, 根据系统的时序收敛问题确定系统的关键时序^[47]。这样有助于降低后续设计任务的难度, 提升集成电路的设计效率。

(3) 逻辑设计。用文本和原理图描述功能设计阶段子模块的逻辑结构, 通过模拟仿真验证该逻辑结构的正确性并对结构进行优化。

(4) 电路设计。用逻辑综合工具把数字逻辑转换为详细的电路图, 在此过程中还要考虑到元器件的各种参数、电路的延时和功耗的限制。

(5) 物理设计。物理设计是对逻辑功能的具体实现, 是整个设计流程中最关键的环节, 在下一小节会详细介绍。在物理设计阶段, 所有的模块、门电路、晶体管等都会有具体的几何表示, 把整个电路按照功能分为若干个子电路, 在子电路上进行布局规划并进行连线, 最后通过压缩来减少芯片面积。

(6) 设计验证。在物理设计完成后, 这一步需要保证电气和逻辑功能的正确, 包括原理图对比、设计规则检查、电路网表提取、电气规则检查和寄生参数提取^[48]。

(7) 电路制造。把经过上述处理的最终版图 (通常用 GDSII 布局文件表示) 发送给晶圆厂进行流片。集成电路的制造首先通过成膜、光刻、刻蚀和掺杂将晶体管等器件嵌入到硅片上, 之后用导线把元器件连接起来。

(8) 测试封装。通过封装可以使集成电路内部器件与外部环境隔开, 起到保护内部结构、重新分布引脚和芯片散热的作用, 影响着应用、成本和形成因素需求。封装类型包括小外形封装、双列直插式封装、插针网格阵列封装和球栅阵列封装等。封装之后, 集成电路通过在各种条件下测试其电气特性来验证是否满足设计需求。

在实际的设计工作中, 这些流程不是一次就能达到设计指标, 其中的一些步骤可能经过多次重复才能最终满足系统需求, 流程如图 2.1 所示。

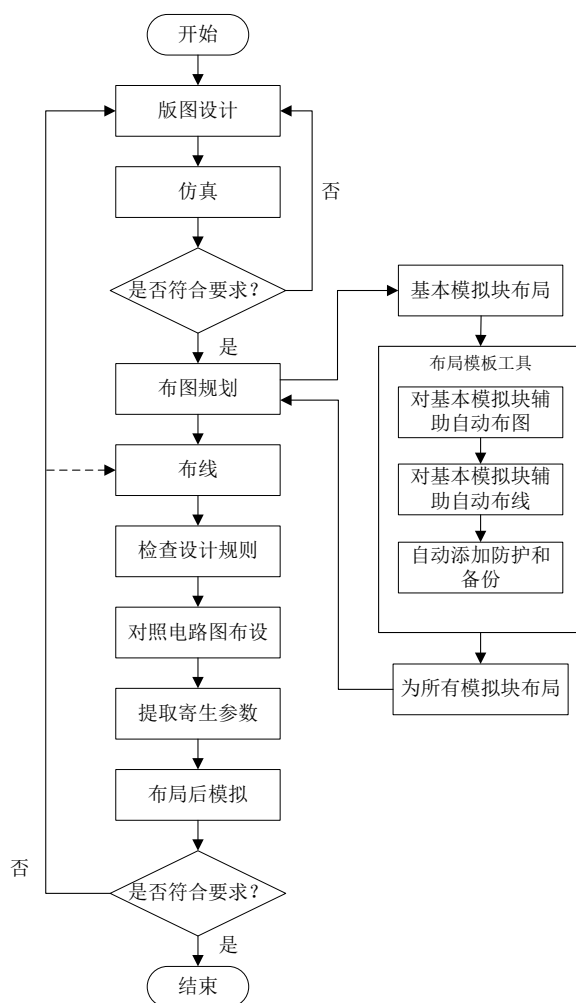


图 2.1 VLSI 设计流程

2.2 物理设计

物理设计也叫做版图设计（Layout Design），其输入是电路设计过程得到的组件和网表的描述，输出是实际的电路几何布局。物理设计在 VLSI 的设计流程中起到了承上启下的作用，也是十分重要的一环。它是一个包括门级网表综合、布图规划、电源规划、布局、布线、时钟树综合和最终创建 GDSII 布局文件的过程，对芯片设计周期、生产成本、性能、面积、功率、可靠性和产量都有很大影响^[49]，物理设计周期不同阶段的简单表示如图 2.2 所示。

2.2.1 电路划分

按照一定标准把整个电路划分为若干个子电路，以这些子电路为主体，对它们进行后续步骤的处理，这样可以降低问题的复杂性和满足封装性。为了解决 VLSI 设计日益复杂的问题，分层设计方法开始广泛使用。图 2.3 为一种分层设计方法。

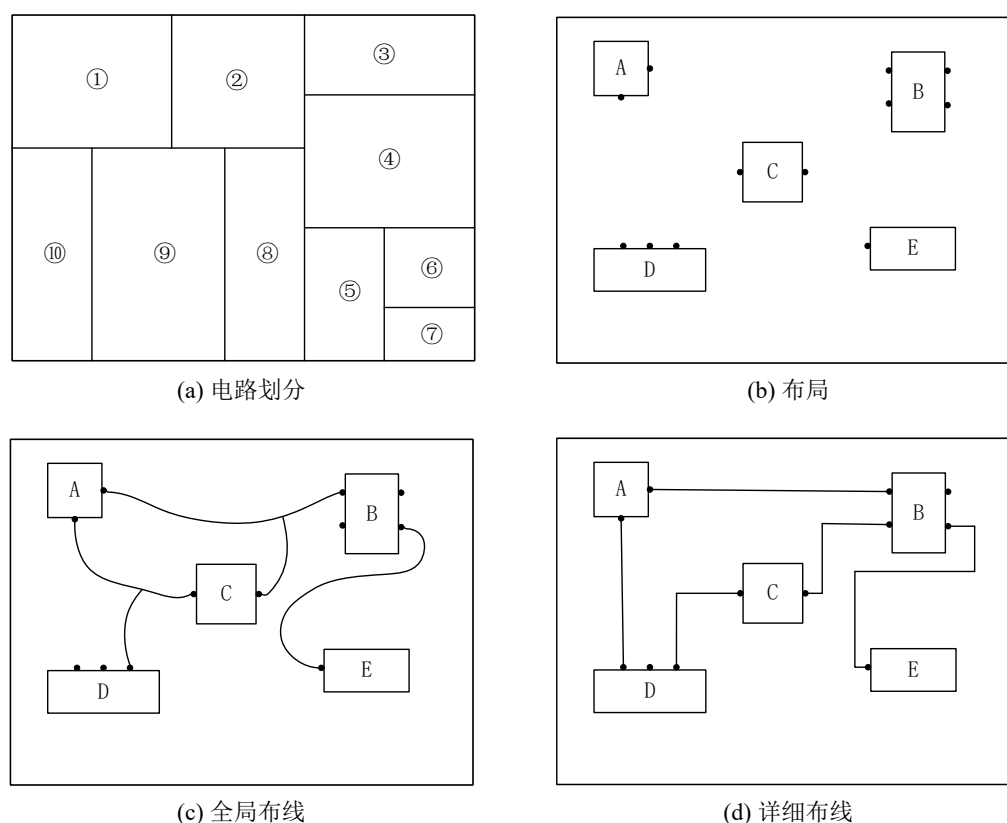


图 2.2 物理设计的不同阶段

2.2.2 布局规划

布局规划是物理设计的基础，它决定了布局和布线的结果^[50]。集成电路的布图模式按照模块的定义可以分为标准单元设计模式（Standard Cell Design Style）、门阵列设计模式（Gate Array Design Style）、门海设计模式（Sea-of-Gates Design Style）、现场可编程门阵列（Field Programmable Gate Array, FPGA）和全定制模式（Full-custom Design Style）。全定制模式中主要有交互图形编辑、符号版图设计方法和积木自动块布图（Block Building Layout, BBL），BBL 模式如图 2.4 所示。在 BBL 模式中，模块可以以任意形状放置在需要的位置，对布线区域的限制较少，更接近人工设计方法，可以应用在自动化设计中^[48]。

布局规划的目的是确定模块在芯片上的最佳位置和最小化芯片总面积，并保证模块之间的连接满足约束，优化标准主要是导线长度、时间延迟、功耗和温度^[51]。

因为模拟信号对制造工艺和工作环境是极其敏感的，所以模拟电路的布局十分复杂，需要对布局设计进行细致的调整。为了最大限度的降低布局对集成电路的影响，可以使用交错结合、器件合并、接壳和防护圈保护等技术，这些技术如何使用主要取决于设计师的经验，经过原理图设计、仿真和物理设计的多次迭代就能满足设计规范。为了缩小高性能模拟电路的电气设计与物理设计的差距，模拟模块中的器件必

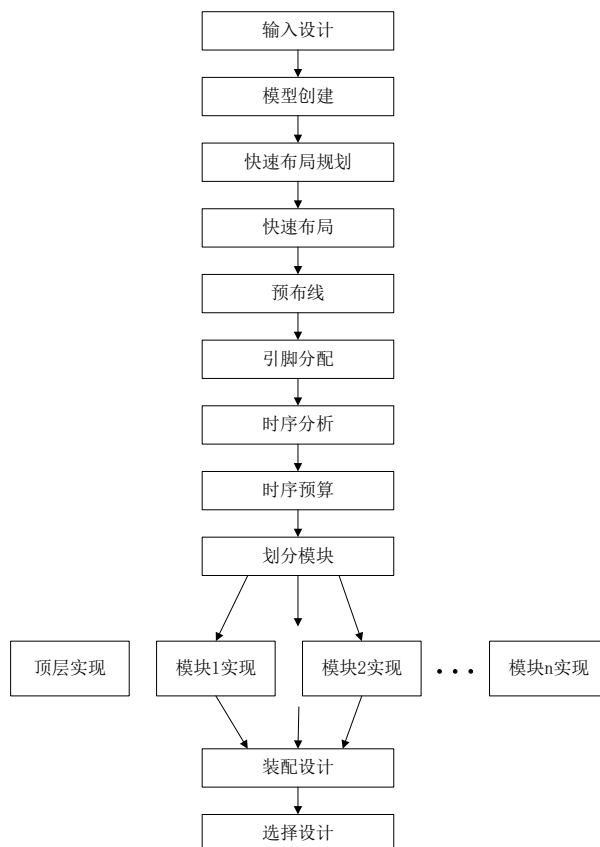


图 2.3 分层设计方法

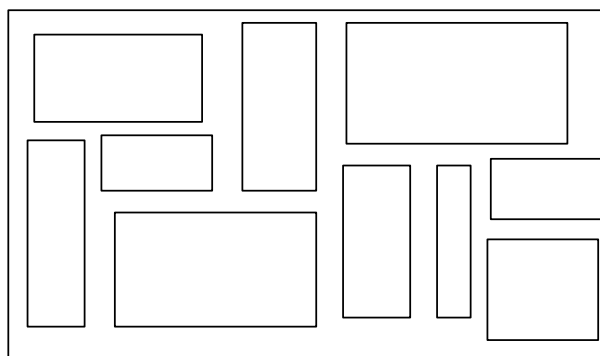


图 2.4 BBL 布图模式

须相互交叉以对称方式放置^[52]。基于工艺梯度、问题梯度、各向异性效应和边界效应的考虑，使用特殊的晶体管布局样式可以最小化器件失配。交错放置和公共质心放置被证明能够有效降低因芯片上参数发生线性变化而导致的失配问题^[53]。

随着芯片的设计规模越来越大，运行速度越来越快，使用的技术越来越先进，芯片的功耗逐渐成为影响芯片性能的重要指标。功耗增大会对芯片的散热性能和封装产生一定影响，使得芯片成本更高，甚至会直接影响到芯片功能的正确性。合理的布局规划、布线和时钟树物理集成是芯片低功耗设计的关键所在，可以使用门控时钟、多阈值电压、多电源电压、电源关断和低功耗时钟树综合的设计方法^[54]。

2.2.3 布线

随着芯片特征尺寸的缩小,互连线的延时已经大幅度超过门电路延时,成为制约芯片性能提升的主要因素。在完成布局规划之后,接下来要通过布线来实现各个模块之间的连接。布线问题可以分为全局布线和详细布线两个阶段,首先在全局布线阶段为每个区域生成粗略的布线,然后在详细布线阶段为指定的布线区域建立实际的几何布局。

以标准单元设计模式来讨论。因为芯片总面积约等于模块面积加上连线占用的面积,所以减少导线长度也可以减小芯片面积。导线长度也是估算布线空间要求、延迟和功耗的一个重要变量,最小化导线长度可以降低电容,从而降低芯片的功耗。由于布线不只是一层上完成的,通孔的数量在线网可布性和时序方面起着至关重要的作用。虽然使用通孔可以减少导线长度,但是通孔本身的电容更高,因此在设计过程中必须保持这导线长度和通孔数量之间的平衡^[55]。减少通孔数量有助于减少布线中金属触点的数量,从而提高电路性能。

2.2.4 压缩

在解决布线问题后,压缩的主要目的是进一步减少芯片面积。布局迁移可以应用分层设计来减少芯片的电路面积^[56],有两种方法可以解决布局迁移问题。第一种方法^[57]根据新的设计规则重新计算布局元素的尺寸,减少元素之间的间隙,执行布局压缩过程;第二种方法^[58]将原始布局描述从某些设计规则的真实形式转换成抽象的虚拟形式,之后借助软件压缩工具把布局调整到所需的设计规则。对于顶层对象的布局压缩,它们设计标准的调整可以由模块布局设计系统“单元匹配”来执行^[59]。

布局描述从真实形式到虚拟形式的迁移提供了布局描述的定性和定量转换,可以用下面三种方式实现:

- (1) 从真实布局层中的几何图形到虚拟层中的图形。
- (2) 从真实单位到虚拟单位。
- (3) 从真实布局描述语言操作符到虚拟布局描述语言操作符。

图 2.5 展示了从真实布局层到虚拟布局层和一些几何图形到其他几何图形的过渡示例,左边为初始布局草图,右边为最终布局草图。

在图 2.5 中,真实的布局描述到虚拟的布局描述的迁移假定了几何图形的转换,用几何图形来描述连线。这是因为在单元描述的实际布局中,连线通常会给出其边界顶点的坐标,此类连线的宽度在压缩时可以减小到设计规则允许的最小导线宽度,当然这也会改变导体的电阻。在这种移植过程中,从多边形对连接的描述过渡到轨迹线对连接的描述,其中轨迹线的特征是每个线段或整条轨迹线的宽度恒定。

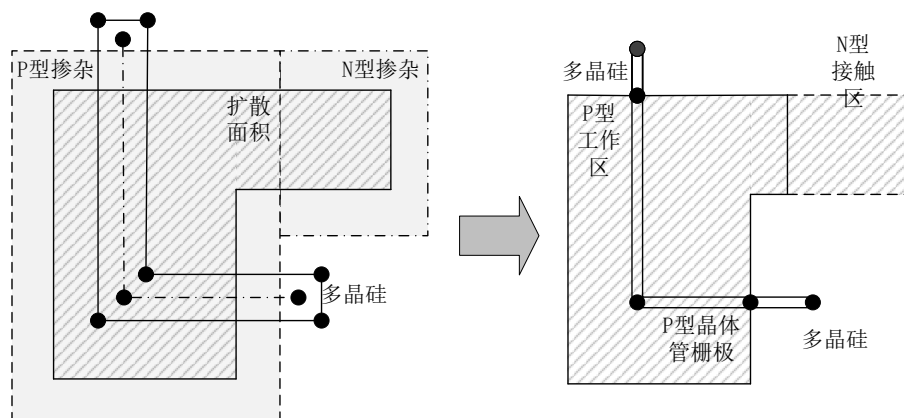


图 2.5 布局草图的过度

2.3 全局布线

正如图 2.2 展示的全局布线过程，全局布线的目标是给出线网的大致走向而不会给出具体的走线结果，它是详细布线的基础。全局布线的质量影响芯片的时序、功率和密度，因此全局布线是物理设计中非常关键的阶段^[60]。

全局布线的所有情况均为 NP-hard 问题^[61]，已经有许多学者为其开发了多种启发式算法。这些算法总体上可以分为顺序全局布线和并发全局布线。顺序全局布线首先根据线网的重要性对线网进行排序，然后按照线网的优先级进行布线。优先级低的线网在布线过程中可能被之前的线干扰，因此需要通过拆线重布或其他方法解决。顺序启发式技术不能确定是否存在可行的解决方案，换言之，如果没有找到可行解，不清楚这是因为不存在可行解还是因为所用算法的不足导致无法找到可行解；在能找到可行解的情况下，不知道这个解是否最优，也不知道得到的解离最优解有多远。为了避免线网排序问题并使解决方案可以预测，以整数规划模型形式的基于并发的全局布线算法被开发出来，此类算法可以同时对所有线网布线。在数学规划的方法中，全局布线被公式化为 0-1 整数规划问题，将它与斯坦纳树问题相结合，解决问题的目标是从斯坦纳树集合中找到一个既满足布线要求又最小化布线成本的斯坦纳树。这种方法倾向于产生全局的解决方案并且不需要线网的初始排序。

2.4 详细布线

在全局布线的基础上，详细布线需要给出要连接器件的具体线路。在双层布线中，详细布线分为两边通道布线、开关盒布线和 L 型通道布线。如果线网端点能出现在矩形边界的四条边上，那么这类布线问题叫做开关盒布线；如果端点只能出现在两条平行的边上，那么此类布线问题叫做两边通道布线，通道的两条边拥有相同的列数。

通道布线是 VLSI 物理设计中的一个关键问题，它的主要目标是在布线过程中尽可能减少芯片的面积。如果能减少通道数目，芯片面积自然会减小。复杂线网在单层布线过程中不可避免的会产生交叉，这种情况需要考虑加入更多层来拓展布线空间。多层布线的总体要求是在实现布通率 100% 的前提下对通孔数目和布线路径的综合优化，虽然通孔能减少导线长度，但通孔自身的电阻也会对电路性能产生影响，如果芯片上通孔数量过多，芯片的成品率就会降低。以双层布线模型为例，曼哈顿架构中垂直线和水平线分别放置在不同层，这里用实线和虚线区分，选择其中一层作为主布线层，在垂直方向布线，另外一层为辅助布线层，在水平方向布线。两边通道布线和开关盒布线的示例分别如图 2.6(a) 和图 2.6(b) 所示。

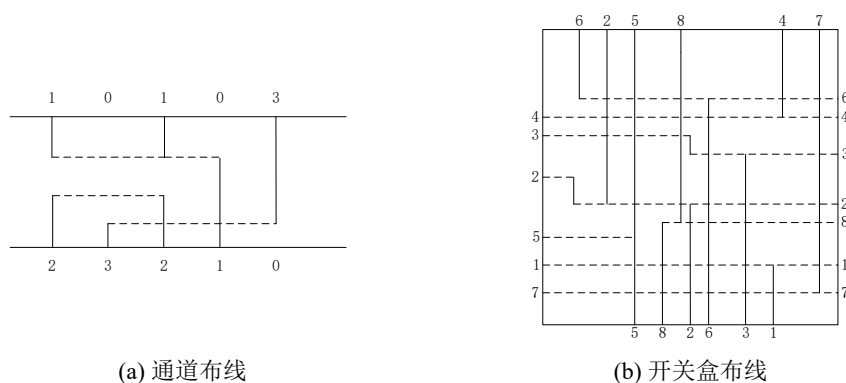


图 2.6 两种布线方式

在 BBL 布图模式中，详细布线需要对布线区域进行划分和排序。详细布线可以看作一个规划问题，总体目标是实现所有线网的连接，子目标是连接每个线网的所有引脚。这些子目标相互依赖，相互竞争资源。子目标的顺序可以由线网重要性和区域内线网数量确定。在图 2.7 的例子中，为了把四个通道都布通，可以先布通道 1，为了布通道 1 则需要先布通道 2，而布通道 2 需要先布通道 3，布通道 3 又需要先布通道 4，布通道 4 反过来还需要布通道 1，这样就形成了一个死循环，不同通道相互制约。

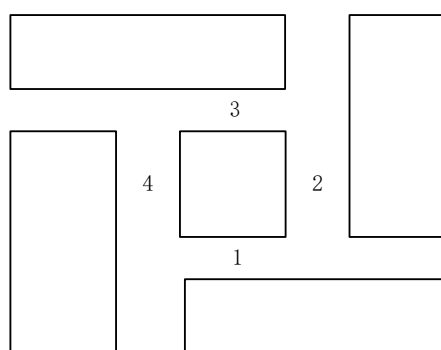


图 2.7 通道布线问题

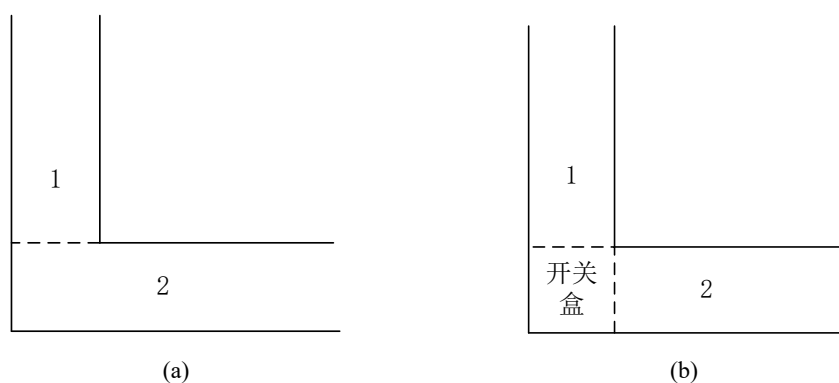


图 2.8 通道问题分解

针对通道互相影响的问题可以把 L 型通道拆分为若干个子区域，子区域由两边通道和开关盒组合而成。图 2.8(a) 把 L 型通道分解成 2 个两边通道，图 2.8(b) 把 L 型通道分解成 2 个两边通道和 1 个开关盒。

布线区域的拓扑结构决定了详细布线问题的复杂程度。单层详细布线问题通常是 NP 完全问题^[62]。即使有再简单的形状，多层的详细布线问题也是 NP 完全问题^[63]。

2.5 本章小结

本章首先介绍了 VLSI 的整体设计流程，着重介绍了流程中最重要的物理设计。物理设计在 VLSI 设计流程中起到承上启下的作用，由电路划分、布局规划、布线和压缩这四个步骤组成。电路划分可以减小电路规模、简化问题；布局规划可以确定模块的位置保证芯片面积最小化；布线可分为全局布线和详细布线，全局布线实在完成布局规划后实现线网分配，确定线网的大致走向，全局布线的质量影响芯片的时序、功率和密度；详细布线是在全局布线线网分配的基础上，给出各线网的具体走线。基于布线问题的复杂性，可以引入启发式智能优化算法来提高解决此问题的能力；最后一步压缩可以进一步减小芯片面积，降低成本并提高芯片性能。

第三章 基于蚁群算法的布线研究

3.1 蚁群算法的介绍

3.1.1 蚁群算法的原理

研究发现,蚂蚁的视觉能力是很差的,它们的活动主要依靠一种叫做信息素的分泌物。对于一只在岔口的蚂蚁来说,它很有可能是随机移动的,移动过程中会在走过的路径上释放信息素,信息素可以被其它蚂蚁个体感知进而获取信息。蚁群算法参照了现实中蚂蚁觅食和蚁群间的交流过程。蚂蚁会在搜索过程中留下信息素,信息素也会随着时间挥发,研究表明信息素浓度最大的路径总是更接近最佳路径。当蚂蚁选择下一步要去的位置时,它们更倾向于信息素浓度高的路径^[64]。经过蚁群的不断强化,较短路径上信息素浓度会越来越大,更多的蚂蚁会被吸引到该路径上,最终经过不断循环可以找到最短路径。

蚁群觅食原理如图 3.1 所示,在蚁巢与食物之间存在不止一条路线,靠近中间这条线路是最短的,但是在初始阶段蚁群可能会搜索到更远的路径,随后蚁群在周围空间搜索,在信息素的作用下,蚁群最终会趋向于最短路径。正是由于蚁群算法的全局寻优能力,它可以被应用到优化和搜索问题。

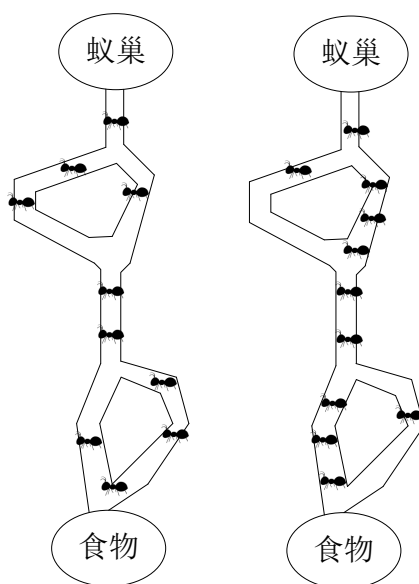


图 3.1 蚁群觅食过程

蚁群算法具有以下优点:

(1) 分布式计算。在蚁群算法中,任意个体都可以独立进行搜索,并把搜索信息和结果共享给整个蚁群,某个单独的个体在搜索过程中出现问题并不会影响最终结

果，这样增加了算法的可靠性和全局搜索能力。通过对所有蚂蚁搜索结果的对比总会找到一个最优解^[65]。

(2) 正反馈机制。在算法的初始时刻图中信息素浓度都是一样的，当一只蚂蚁走过一条路径后全图信息素就会产生差异。蚂蚁所寻找到的路径越短，该路径上的信息素浓度也就越高，这样会吸引其他蚂蚁在这条路径上移动，使得此路径信息素浓度更高，更能吸引蚂蚁，最终可以找到最短路径。这个过程体现了蚁群算法的正反馈机制。

(3) 自组织性。在迭代初期，单个的蚂蚁会随机寻找路径，在信息素的作用下，经过一段时间的迭代蚁群会自发的趋向接近最优解的路径，此过程不需要外界的干预。

(4) 鲁棒性。蚁群算法对初始路线的要求不高，参数少，在搜索过程中不需要人工进行调整。只需要对算法进行少量改进就能解决不同问题。

3.1.2 蚁群算法的实现过程

在 TSP 中，给定 n 个城市，需要规划出一个从某个城市出发，把其他城市都各访问一次，最终返回出发城市的最短路线。TSP 是一个典型的 NP 问题，它可以抽象为一个有向加权图 $G = (N, A)$ ， N 表示城市的节点集， A 表示边集，每个边 $(i, j) \in A$ 都有一个权值（距离） d_{ij} 。

设蚁群中有 m 只蚂蚁，对于蚂蚁 k 从城市 i 移动到城市 j 的概率公式为：

$$P_{ij}^k = \begin{cases} \frac{\tau_{ij}^\alpha \times \eta_{ij}^\beta}{\sum_{j \in allowed_k} \tau_{ij}^\alpha \times \eta_{ij}^\beta}, & \text{如果 } j \in allowed_k \\ 0, & \text{否则} \end{cases} \quad (3-1)$$

其中， τ_{ij} 为城市 i 到城市 j 路径上的信息素浓度； α 表示信息素启发因子，它的值越大代表蚁群的正反馈作用越大； $\eta_{ij} = 1/d_{ij}$ 为启发函数， d_{ij} 表示城市 i 与城市 j 之间的距离； β 表示启发函数重要因子，它的值越大代表蚁群的选择越贪婪； $allowed_k$ 表示蚂蚁 k 还没有访问过的城市集合，初始阶段 $allowed_k = N$ ，蚂蚁每访问一个城市就会把这个城市从集合中删除，当集合为空就表示蚂蚁搜索完成。

蚂蚁从起始点移动到终点的过程中会在走过的路径上留下信息素，然而之前所有路径的信息素也会不可避免地挥发，在蚂蚁 k 移动到终点后需要对路径上的信息素进行更新，信息素更新公式如下所示：

$$\tau_{ij} = (1 - \rho)\tau_{ij} + \Delta\tau_{ij} \quad (3-2)$$

其中，参数 ρ 为信息素挥发因子，为了避免信息素的无限积累，图中所有边上的信息素会呈指数下降，使得算法能够在迭代过程中舍弃错误路径。每次迭代后信息素

τ 会挥发掉 $\rho\tau$, 其取值满足 $\rho \in (0, 1)$ 。 $\Delta\tau_{ij}$ 表示所有蚂蚁从城市 i 移动到城市 j 在路径上留下的信息素之和, 计算方式如下:

$$\Delta\tau_{ij}(t) = \sum_{k=1}^m \Delta\tau_{ij}^k(t) \quad (3-3)$$

其中, $\Delta\tau_{ij}^k(t)$ 表示蚂蚁 k 从城市 i 移动至城市 j 在路径上留下的信息素, 信息素变化值的大小根据不同模型有不同的计算公式^[1], 分别是蚁周模型 (ANT-cycle)、蚁量模型 (ANT-quantity) 和蚁密模型 (ANT-density), 计算公式如下所示:

(1) 蚁周模型:

$$\Delta\tau_{ij}^k(t, t+n) = \begin{cases} \frac{Q}{L_k}, & \text{如果第 } k \text{ 只蚂蚁在本次迭代过程经过边 } (i, j) \\ 0, & \text{否则} \end{cases} \quad (3-4)$$

(2) 蚁量模型:

$$\Delta\tau_{ij}^k(t, t+1) = \begin{cases} \frac{Q}{d_{ij}}, & \text{如果第 } k \text{ 只蚂蚁在 } t \text{ 到 } t+1 \text{ 时间经过边 } (i, j) \\ 0, & \text{否则} \end{cases} \quad (3-5)$$

(3) 蚁密模型:

$$\Delta\tau_{ij}^k(t, t+1) = \begin{cases} Q, & \text{如果第 } k \text{ 只蚂蚁在 } t \text{ 到 } t+1 \text{ 时间经过边 } (i, j) \\ 0, & \text{否则} \end{cases} \quad (3-6)$$

其中, Q 表示信息素, 为一个常数; L_k 表示蚂蚁 k 搜索到的路径长度; d_{ij} 表示城市 i 和城市 j 之间的距离。从公式中可以看出在蚁密模型中, 当蚂蚁从城市 i 移动到城市 j 时, 边 (i, j) 信息素增加的大小与 d_{ij} 无关, 在蚁量模型中与 d_{ij} 成反比。

在蚁周模型中的 $\Delta\tau_{ij}^k$ 是在蚂蚁迭代结束后才进行更新, 而其他两种模型是蚂蚁移动后立即进行更新, 本文使用的是蚁周模型。蚁群算法解决问题的步骤如下:

步骤 1: 设定算法中的参数, 包括信息素启发因子 α 、启发函数重要性因子 β , 信息素挥发因子 ρ , 蚂蚁种群数量 m , 信息素 Q 和最大迭代次数 Nc_{max} , 初始化禁忌表和路径长度。

步骤 2: 对图进行初始化, 除起点外将图中各个点设为未访问过的点, 把各边的信息素浓度设为一个固定值, 初始状态下禁忌表只有起点坐标。

步骤 3: 根据概率选择公式计算出当前点可以到达的各点概率并选择向其中一点移动, 将该点加入禁忌表中, 更新当前路径长度。

步骤 4: 当蚂蚁到达终点后将当前搜索到的路径与之前路径比较, 选择更好的路径作为备选答案。如果当前所得解没有收敛而且迭代次数未达到最大值, 那么就更新图中各边的信息素, 转向步骤 2 继续搜索, 否则进行下一步。

步骤 5: 算法结束, 将得到的最好结果输出。

蚁群算法解决 TSP 问题的流程如图 3.2 所示:

升。如果发现算法在搜索过程中已经陷入局部最优，可以把搜索空间重新初始化来跳出局部最优。

所有改进的蚁群算法都有一个重要的共同点：它们比普通蚁群系统更能利用搜索过程中找到的最佳解决方案。此外，它们使用局部搜索的结果来改进蚁群构建的解决方案，对较好解决方案的深入利用是提高算法性能的关键所在，这也与组合优化问题的搜索空间有关。

(1) 选择概率的改进

在一般的蚁群算法中，选择概率只与当前点 i 和下一目标点 j 之间的信息素浓度 τ_{ij} 和距离 d_{ij} 有关，对于曼哈顿架构的有网格布线模型来说，如果每次只走一格的话， $d_{ij} = 1$ 是恒成立的，这样会导致启发函数重要性因子 β 不能发挥作用。然而布线问题不同于现实生活中蚂蚁寻找食物的过程，布线的起点和终点都已知的，那么可以把启发函数改为 $1/d_{ej}$ ， e 代表终点。这样蚂蚁在寻路过程中像是会被终点吸引一样，下一可行点 j 距离终点 e 越近，它被选择的概率就越大。优化选择概率后的算法可以避免蚂蚁在搜索过程中的局部停滞，提升搜索速度，最终获得更好的搜索路径，改进后的概率选择公式如下所示：

$$P_{ij}^k = \begin{cases} \frac{\tau_{ij}^\alpha \times d_{ej}^{-\beta}}{\sum_{j \in allowed_k} \tau_{ij}^\alpha \times d_{ej}^{-\beta}}, & \text{如果 } j \in allowed_k \\ 0, & \text{否则} \end{cases} \quad (3-7)$$

(2) 最大最小蚁群系统

在普通的蚁群算法中，每只蚂蚁在它经过的路径上都会释放信息素，在较大的测试用例上应用时，蚁群算法给出的解决方案质量会非常差。在 MMAS 和基于排名的蚁群系统 (ASrank) 中^[67] 优化了信息素和路径的更新规则。在 MMAS 中，规定每次迭代后只允许找到最优解的蚂蚁更新路径从而保证解的质量；对于基于排名的蚁群系统，蚂蚁在当前迭代中排名越高，它对路径更新的权重越大，而全局最优蚂蚁可以更新路径上的信息素。MMAS 可以将搜索过程中发现的最优解改进利用，与避免早期搜索停滞的有效机制相结合来得到更好的解。

最大最小蚁群系统与普通蚁群算法的主要区别在于以下几点：

普通蚁群算法可以通过只允许本次迭代周期找到最优解的蚂蚁更新信息素来优化。但这种方式有一个不足之处是会造成算法的早期停滞，这个问题会在下一点得到改进。当停滞问题发生时，某些路径上的信息素相较于其他路径会非常高，导致蚂蚁只会在这些路径上一再重复。而在最大最小蚁群系统中，找到最优解的蚂蚁才允许更新或修改路径信息^[68]。

一方面，在迭代完成之后更新信息素的时候，某条路径上的信息素可能高于其他路径很多，蚂蚁更有可能选择此路径；另一方面，一些路径上的信息素会因为蒸发而

趋近消失，导致搜索空间不足。这两种情况都会使得蚁群系统迅速收敛到较差的解决方案上。MMAS 引入了两个参数，即信息素浓度最大值 τ_{max} 和信息素浓度最小值 τ_{min} ，通过将路径上的信息素限制在 $[\tau_{max}, \tau_{min}]$ ，有效缓解了搜索停滞问题，这也是最大最小蚁群系统名字的由来。

在算法的初始化阶段，将信息素的初值设定为 τ_{max} ，信息素挥发因子 ρ 设为较小值，这样可以提高算法初始阶段的搜索效率。如果把初值设为 τ_{min} ，随着迭代的进行，不同路径信息素之间相对差异会大大增加，从而失去了优化的作用^[69]。改进后的信息素更新公式为：

$$\tau_{ij} = (1 - \rho)\tau_{ij} + \Delta\tau_{ij}^{best} \quad (3-8)$$

$$\Delta\tau_{ij}^{best} = 1/f(p^{best}) \quad (3-9)$$

其中， $f(p^{best})$ 表示每次迭代的最优蚂蚁 p^{ib} 或者全局最优蚂蚁 p^{gb} 的解决方案成本。根据实验证明^[69]：如果选择使用全局最优蚂蚁 p^{gb} ，蚁群的搜索会很快收敛到最优蚂蚁的解周围，很可能因为搜索空间过小而收敛到局部最优解；如果使用迭代中的最优蚂蚁 p^{ib} ，由于每次迭代的最优解可能会相差很大而且经过大量的迭代后最优解才偶尔会得到强化，这样收敛到局部最优解的可能性就降低了。

在本文中使用的 τ_{max} 和 τ_{min} 如下所示：

$$\tau_{ij}^{max} = \frac{1}{\rho \times f(p^{ib})} \quad (3-10)$$

$$\tau_{ij}^{min} = \frac{\tau_{ij}^{max}}{\mu \times g_{\tau}} \quad (3-11)$$

其中， μ 表示最大最小蚁群系统所用蚂蚁的数量， g_{τ} 是决定信息素最大值相对大小的系数。在更新信息素时，如果 $\tau_{ij} > \tau_{max}$ ，则 $\tau_{ij} = \tau_{max}$ 或者 $\tau_{ij} < \tau_{min}$ ，则 $\tau_{ij} = \tau_{min}$ 。

3.2 布线模型的路径图

3.2.1 集成电路布线模型

集成电路根据架构的不同，它的布线规则 and 性能表现也有所不同。除了传统的曼哈顿架构外还有很多其他的架构模型，例如 X 架构^[70] 和 Y 架构（也叫做六边形架构）^[71]，两种架构的网格如图 3.3 所示。不同架构的布线结果有很大不同，图 3.4 为曼哈顿架构和 X 架构的布线结果。

在集成电路的物理设计流程中，布线分为全局布线和详细布线。在全局布线阶段，布线区域被分成大小不一的网络，并把线网分配到合适的网络中以保证布通率尽

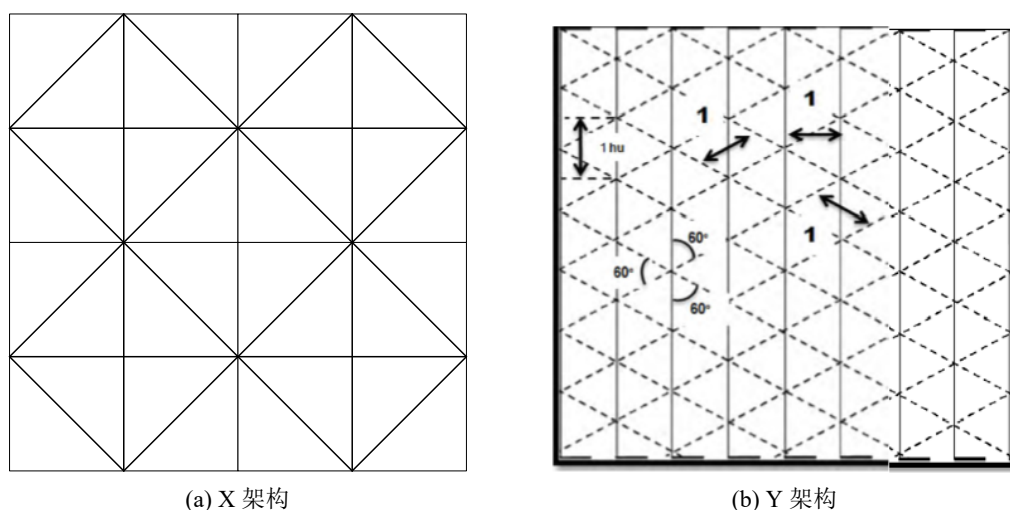


图 3.3 不同架构布线模型

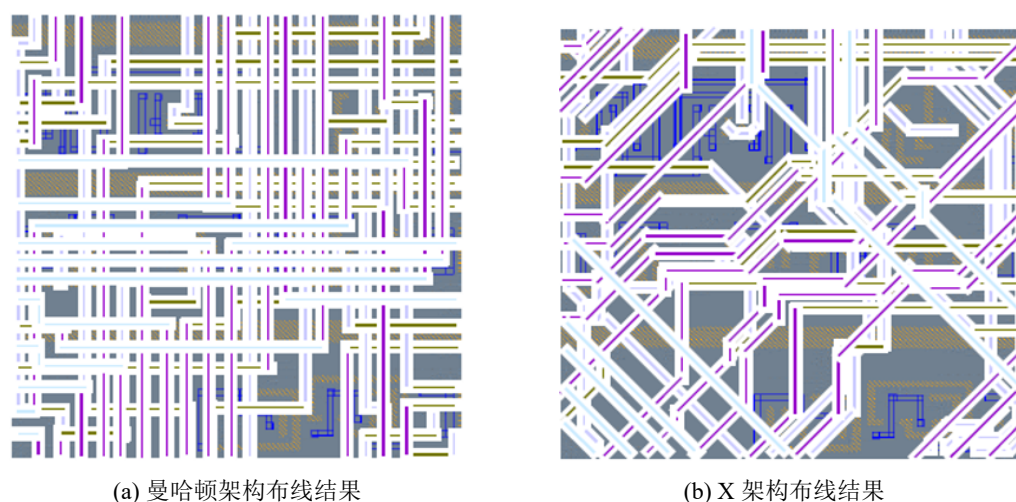


图 3.4 不同架构布线结果

可能高，在此阶段会为每个网络生成粗略的布线结果但不会指定导线实际的几何布局。在详细布线阶段则会确定各个网络的具体走线。

布线问题使用的模型一般分为有网格模型和无网格模型。对于有网格模型来说，连线两端都必须在网格点上，连线本身也必须在网格线上。有网格布线的实现过程相对简单，算法研究也很成熟，1961 年提出的迷宫算法已经被广泛应用到有网格布线器算法当中。有网格模型的布线区域被分成均匀的网格，布线图的精准程度取决于网格本身的大小，若需要很高的布线精度，网格需要很小，这会导致问题规模的增加。在布线过程中，网格会根据导线的线宽和线间距的要求进行设定，但这种划分方式会导致电路板的有效布线区域小于实际布线区域。假设网格的边长为 5 个单位，把一个边长为 6 个单位的正方形器件放置在网格图上会占据 9 个相邻的网格，被占据的 9 个网格不能进行布线，若网格大小变为 2.5 个单位，那么占用的面积就更小，如图

3.5 所示。网格缩小使同样面积的网格区域中网格数量呈指数倍增加，计算所需要的内存和时间也同样如此，所以有网格布线不适合解决大规模布线问题。对于集成电路来说电路板应当只受制于加工精度，应该是无网格的。对于有网格布线可能出现一种情况：线网本身是可以布通的，但网格的使用导致本来可以布通的线网不能布通，或者能布通但走线很不合理。

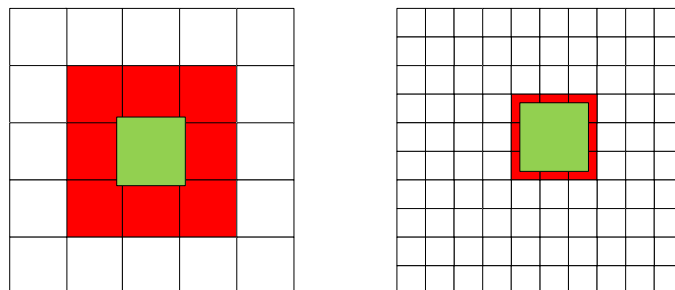


图 3.5 器件在不同大小网格的影响

1984 年诞生了世界上第一个无网格布线器^[72]。无网格模型仅把电路板看作元器件和导线等形状的集合，布线区域被分为大小不一的矩形，其复杂程度与元器件的复杂程度和数量有关。在无网格模型中，线网和元器件可以任意放置而不必限定在网格上，导线的线宽和线间距也可以自由变化，这是有网格模型不能实现的。无网格模型相较于有网格模型可以更有效地利用布线资源，更加贴近实际情况，增加了布线的灵活性，线网可以更加紧凑，后续布线就有更大空间，也有利于提高布通率和减小芯片面积；而且在布线过程中不必存储每个网格的数据，所以无网格模型所用内存和时间都少于有网格模型。下面使用无网格模型来解决集成电路布线问题。

3.2.2 路径图模型

无网格模型没有网格作为路径搜索的依托，它需要先通过一些方法预先找到布线区域可以使用的节点。一种解决方法是 1969 年 Hightower 提出的线探索法，它以线段为单位进行搜索，第一步是基于深度优先搜索的线搜索过程，第二步是基于广度优先搜索的线拓展过程^[73]。线探索法可以减少布线算法的时间和空间复杂度，使路径的拐点数目最小但不能保证找到最短路径。另一种方法是基于图论的算法，把双端布线问题转换为图论中寻找两点间最短路径的问题，这样布线问题就可以与图论的理论相结合来建立路径图。将蚁群算法与无网格模型相结合，利用蚁群的群体优化策略可以在较少的搜索空间找出理想结果。

对单层布线来说，设布线平面为 R ，布线区域的障碍为 O ，则 $O = \{O_1, O_2, \dots, O_k\}$ 组成了非布线区域， $G = R - O$ 为可布线区域。设 s 为布线起点， t 为布线终点，线网端点集合为 $V = (s, t)$ ，如果可布线区域 G 中存在子集 S ，满足 s 和 t 可以在 S 中连通，即 S 中存在至少一条 s 到 t 的路径，则称 S 为 V 的连接图 (Connection Graph)。若能

在 S 中找到 s 到 t 的最短路径, 那么 S 为 V 在 G 中的强连接图 SC (Strong Connection Graph)^[74]。如果可以在 G 中找到 SC 来代替 G , 且 SC 的规模比 G 的规模小很多的话, 就可以简化问题, 降低搜索的复杂度。在曼哈顿架构中, 两个点 (x_i, y_i) 和 (x_j, y_j) 之间的距离是用曼哈顿距离 MD 描述的, 即 $MD = |x_i - x_j| + |y_i - y_j|$ 。如果知道两点的坐标, 那么两个点的距离就可以计算出来。如果布线区域没有障碍的话, 两点间的最短路径就等于两点间的曼哈顿距离; 如果布线区域有障碍的话, 可以用障碍和线网端点构造布线的路径图, 以此实现简化问题、节省存储空间和提高布线速度的目的。

在一个布线区域中, 给定线网端点集合 $V = (s, t)$ 和障碍 $O = \{O_1, O_2, \dots, O_k\}$, 其中障碍不会相互重叠且障碍的边由水平和垂直线段组成。双端布线问题可以看作是在一定约束条件下寻找两点之间最优路径的问题, 约束条件可以是路径长度最短、拐点最少或者距离和拐点数目组成的成本函数的值最小。如果只约束路径长度, 则称为最短路径问题 (Shortest Path, SP); 如果只约束成本函数的值, 则称为最小费用路径问题 (Minimum-Cost Path, MCP)^[75]。为了使布线可以利用图论的相关知识, 引入了路径图的概念。因此需要引入以下定义。

定义 3.1 设平面上的障碍由大小不一的矩形拼接而成, 如果障碍其中一条边的两条邻边位于该边的同侧, 就称这条边为极限边, 极限边的端点叫做极限点。

极限边和极限点是计算路径图的重要依据。在起点 s 和终点 t 上沿着上、下、左、右四个方向作射线。终点 t 发出的射线可能遇到以下情况: 与极限边所处的直线相交、与障碍的非极限边相交或者不与障碍相交。通过上面方法得到的线段叫做目标线: 从起点 s 出发的四条射线, 如果相交于极限边所处的直线, 所得的交点用集合 PE 记录, 如果相交于障碍的非极限边, 所得的交点用集合 PB 记录。然后从交点集 PE 和 PB 中的点出发, 如果点不在障碍边上, 继续沿原射线的方向延伸; 如果点在极限边上就沿着极限边向外延伸; 如果交点不在非极限边上就向它的两个端点移动, 这样可以得到 PE 和 PB 的交集或者与目标线相交。继续执行上述步骤直到没有交点, 这样可以得到图 G_0 。根据线探索法的原理, 点 s 到点 t 的最短路径不一定存在于 G_0 中。然后将所有极限边延伸直到遇到图的边界或者障碍, 对点 s 和点 t 发出的射线按照同样的方式处理。将这些交点、射点和它们之间的线段构成图 G ^[76], 这就是布线问题需要的路径图。

定义 3.2 一个平面上有 k 个互不重合的凸规则障碍 $O = \{O_1, O_2, \dots, O_k\}$ 以及起点 s 和终点 t 。障碍 O_k 的极限边沿所在直线延伸直到与其他障碍边相交, 这样得到障碍 O_k 的轨线。从起点 s 向上做射线, 射线与障碍点或障碍的轨线相交得到线段 SS' 。如果 S 位于水平或垂直的轨线上, 或者射线能一直延伸到图的边界, 则 $S = S'$ 。按照这样的方式对起点 s 的其他三个方向的射线进行处理, 对终点 t 也按照 s 点这样进行操作。最后由轨线、障碍边界和它们的交点组成的路径图, 记作 $G(O, s, t)$ 。该方法构

造出的路径图包含从起点 s 到终点 t 的最短路径^[76]。

3.2.3 路径图的构造

上一小节中的轨线是构造路径图的基础,对于图中的每个障碍,首先需要找到障碍的水平和垂直轨线,然后生成线网端点的轨线,最后由它们来形成路径图。

水平轨线和垂直轨线的生成方式是类似的,此处仅讨论垂直轨线的生成,如图 3.6 所示。首先把垂直的障碍边单独列出,从左向右进行扫描,扫描线如果与障碍相交会在障碍内部截出一条线段,称之为活动线段。把扫描出来的线段按照从下往上的方向进行排列,用树形结构 T 来存储活动线段, T 的每个节点保存线段上下端点的纵坐标。设扫描线经过障碍的一条垂直边 e , y 坐标为 $f[y_1, y_n]$ 。按照下面的方式对边 e 进行操作^[75]:

- (1) 如果 e 是左极限边,从 T 中找出一条线段 e_d 低于 e 和另一条线段 e_u 高于 e 。那么由 e_d 的上端点和 e_u 的下端点可以形成一条垂直轨线,将 $e[y_1, y_n]$ 插入 T 中。
- (2) 如果 e 是右极限边,则 T 中也可以找到低于 e 的 e_d 和高于 e 的 e_u 。由 e_d 的上端点和 e_u 的下端点可以形成一条垂直轨线,将 $e[y_1, y_n]$ 从 T 中删除。
- (3) 如果 e 不是极限边,那么 e 用该障碍的活动线段代替。

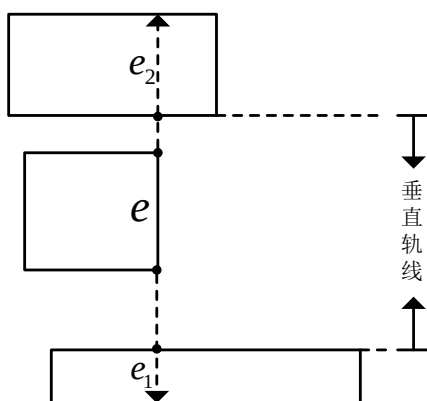


图 3.6 垂直轨线的构造过程

在寻找点 s 和点 t 的垂直轨线时,可以把障碍的水平轨线看作由线组成的障碍,这样通过上面的处理方式就能得到它们的垂直轨线。障碍、线网端点和轨线组成图 $G(V, E)$, 图的顶点 V 由轨线交点和障碍顶点组成, 图的边 E 由轨线和障碍边端点之间的线段组成^[77]。用图 3.7(a) 的例子来说, 图中给出了一些障碍和线网端点。对每一个障碍, 从障碍的各个顶点开始进行水平和垂直扫描, 遇到其他障碍或者图的边界停止, 把扫描的线段存储起来, 对点 s 和点 t 也进行上述的扫描。所有障碍和点扫描完毕后, 把存储的线段进行比较, 找出线段之间的交叉点保存起来, 再加上障碍顶点、线网起始点和扫描线端点。这些交叉点最终构成了图的访问点阵。

无网格模型生成的路径图如图 3.7(b)。图中各线交叉点形成了 $G(V, E)$ 的访问点阵，该访问点阵的规模远小于有网格布线网格点的规模，这样在应用蚁群算法解决布线问题时存储信息素的空间就会减少很多，搜索速度也会更快。

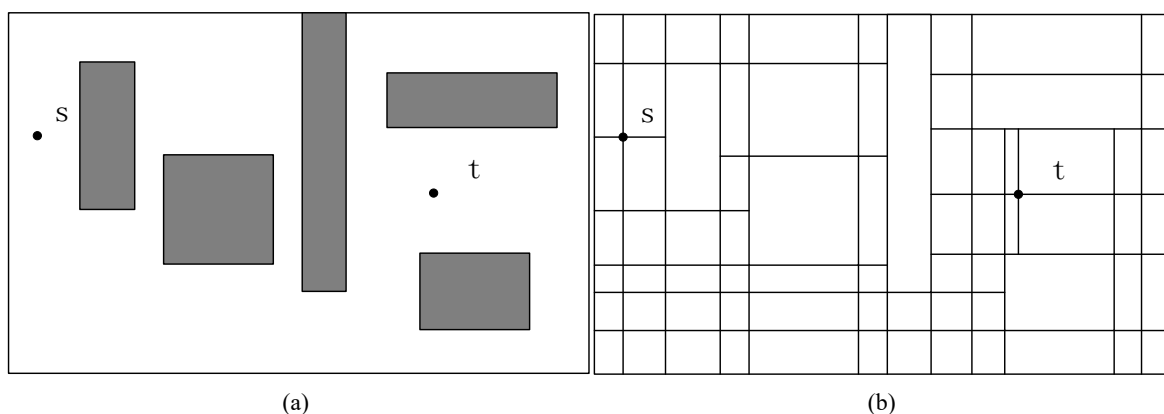


图 3.7 访问点阵的形成

3.3 蚁群算法在布线问题的应用

无网格布线问题与 TSP 有一定的相关性，用蚁群算法解决无网格布线的思路与解决 TSP 是大同小异的，只需要对算法稍加改动就可以应用到布线问题当中。

3.3.1 优化目标

集成电路的连线长度随着电路中晶体管数量的增加而增加，对时间延迟有影响的电容和电阻很难降低而且连线也有一定的面积和宽度，这使得长度成为很重要的参数。导线长度是估算布线空间要求、延迟和功耗的一个重要变量，减少导线长度和通孔数量可以降低芯片电容，从而降低芯片功耗。除此之外，集成电路的电磁鲁棒性对集成电路的可靠性有很大影响^[78]，电迁移现象会产生应力，可以通过限制连线的长度来减少应力从而减少对集成电路的损害。所以集成电路的导线长度是估算布线空间要求、延迟和功耗的一个重要变量^[71]。本文使用的是曼哈顿架构，假设节点 i 的坐标为 (x_i, y_i) ，节点 j 的坐标为 (x_j, y_j) ，则两点间距离 $d_{ij} = |x_i - x_j| + |y_i - y_j|$ 。

布线问题的成本还要考虑到顶点代价函数，如图 3.8 所示，如果在点 v 上连线有弯曲，那么在点 v 上就有代价，否则点 v 的代价为 0。在双层布线模型中，拐点可以转化为多层模型中的通孔。通孔虽然可以有效地减少导线长度但它相较于导线有更大的电容会对电路性能带来不好的影响；通孔还具有影响电路性能的相关电阻；通孔的尺寸通常大于导线宽度，所以通孔数量越多，占据的布线空间就越大；在集成电路中，如果芯片上通孔数量过多，芯片的成品率就会降低，芯片的产量与通孔数量成反

比。路径的总成本由路径长度和路径中拐点的代价构成：

$$f_k = \alpha_l l_k + \beta_v v_k \quad (3-12)$$

其中， f_k 为线网 k 的成本函数， l_k 为线网 k 的路径长度， v_k 为线网 k 的拐点代价函数， α_l 和 β_v 分别为路径长度和拐点代价函数的权重且 $\alpha_l, \beta_v \geq 0$ 。解决布线问题的目标就是使 f_k 的值最小。

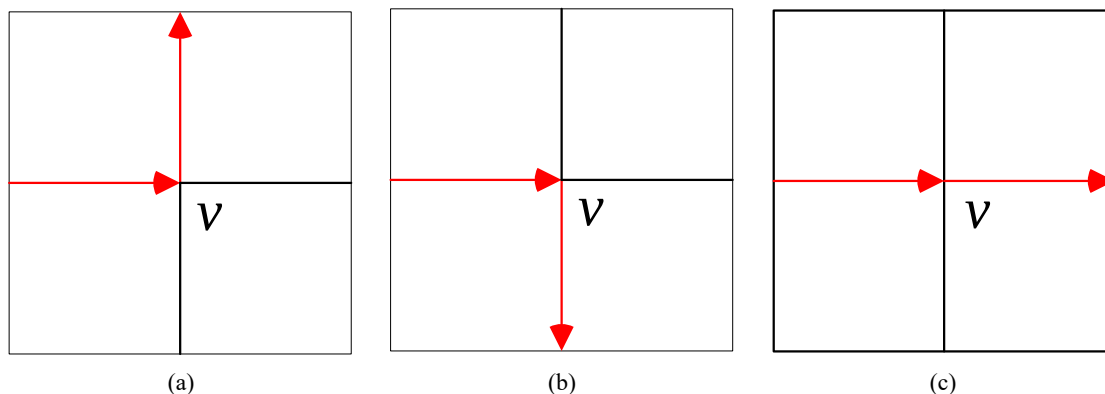


图 3.8 拐点示意图

3.3.2 选择机制

在蚁群算法的实现过程中，可以用轮盘赌法来决定下一步走向哪个点，如图 3.9 所示。轮盘赌法不是根据蚂蚁个体的选择概率，而是根据累积概率来进行选择。假设蚂蚁 k 在当前点 x 有三条路可以走，根据式 (3-7) 分别计算出选择每条路径的概率 $p_1^k(x)$ 、 $p_2^k(x)$ 和 $p_3^k(x)$ ，计算出个体累积概率 $q^k = p_1(x) + p_2(x) + p_3(x)$ ，如下图所示。之后在 $[0,1]$ 区间内产生一个随机数 r ，如果 $r \leq q_1$ ，则选择第一条路；如果 $q_1 < r < q_2$ ，则选择第二条路；如果 $r \geq q_2$ ，则选择第三条路。

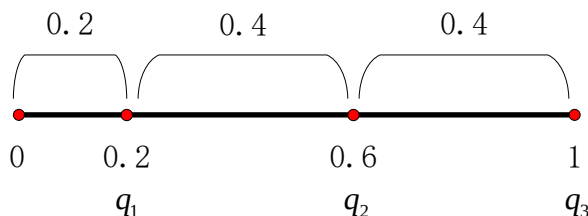


图 3.9 累积概率

3.3.3 蚁群算法用于布线问题的演示

下面用一个简单实例演示蚁群算法解决集成电路布线过程。如图 3.10 所示。布线空间大小为 100×100 ，虚线框表示障碍集合 $O = \{O_1, O_2, \dots, O_k\}$ ，障碍边界也可以走

线，两个蓝点为线网端点。蚁群算法所用参数由表 3.1 给出：

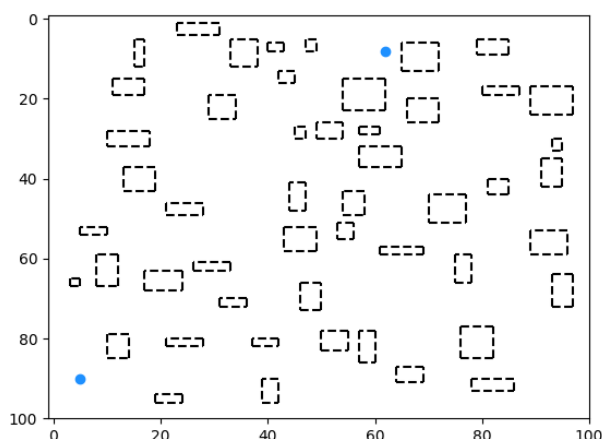


图 3.10 布线图

表 3.1 蚁群算法参数

α	β	ρ	Q	m	Nc_{max}
1.5	5	0.3	100	20	100

这里用有网格布线和无网格布线两种模型进行布线，得到有网格布线结果如图 3.11(a)，无网格布线结果如图 3.11(b)。

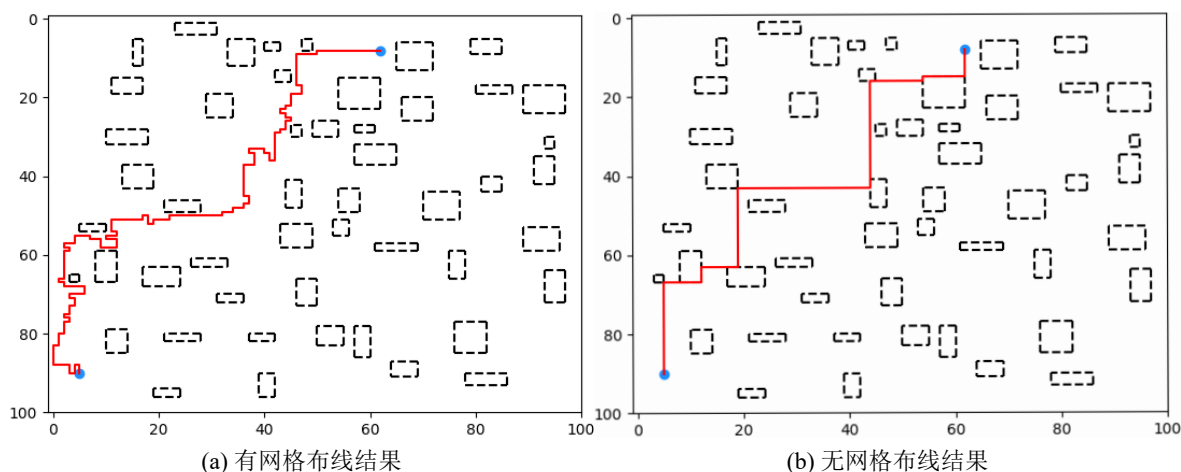


图 3.11 布线结果图

从结果上很容易能够看出来无网格布线的结果不论是路径长度还是拐点成本都要优于有网格布线。蚁群算法的有网格布线在不加额外约束的情况下，每次只走一格且几乎每个节点都有三个岔路可以选择。这样蚂蚁的移动方向很随机，从而导致拐点成本很高，运行时间也远超无网格布线，蚂蚁数量越多、迭代次数越多时间差距越明显。

3.4 本章小结

本章首先对蚁群算法的原理进行介绍，通过求解 TSP 问题给出了蚁群算法的数学模型、实现流程和可以优化的部分。随后给出无网格布线路径图的构造方法使得布线问题得到简化，给出蚁群算法在解决布线问题上的优化目标和选择机制，最后比较了有网格布线和无网格布线的结果，展示了无网格布线的优势所在。

第四章 改进蚁群算法应用于 VLSI 布线

4.1 RFD 算法的介绍

河流形成动力算法（River Formation Dynamics, RFD）是一种基于水滴随机概率运动的启发式方法，是一个与蚁群优化（Ant Colony Optimization, ACO）相关的进化计算方法。粗略地说，RFD 算法可以看作是 ACO 考虑不同自然启发的梯度导向版本，它对于分析 NP 完全问题是很适用的^[79]。对于不同性质的 NP-hard 问题，如旅行商问题^[80] 和最小负荷序列^[81]，实验发现 RFD 算法会在更长的时间内提供良好的解决方案。近年来，有人将 RFD 算法应用于不同类型的优化问题，例如模拟电路规模优化^[82]、无线传感器网络寻找最优数据聚合树^[83] 和移动机器人导航^[84] 等。

4.1.1 RFD 算法的原理

RFD 的思想来自于自然界中水形成河流的过程。假设一些水滴落在平整的地面上，这些水滴会在落点周围进行扩散，直到它们中的一部分找到出口，把最终的出口当作河流最后流入的大海。在流入大海的过程中，当水流经过从高向低的斜坡时，水流通过侵蚀地面、裹挟泥土来改变环境高度，而到达一个平坦的位置时，水流会在此沉积它所携带的沉积物。通过这种方式可以增加或者降低某点的高度并动态地构建梯度下降的路径，梯度差越大的路径越有可能被选择，其他未经侵蚀的区域的水流会更有可能落入这些路径中。通过这种不断强化的方式，水流会在重力的作用下找到从起始点到海洋的路径。RFD 算法的原理示意图如图 4.1 所示：

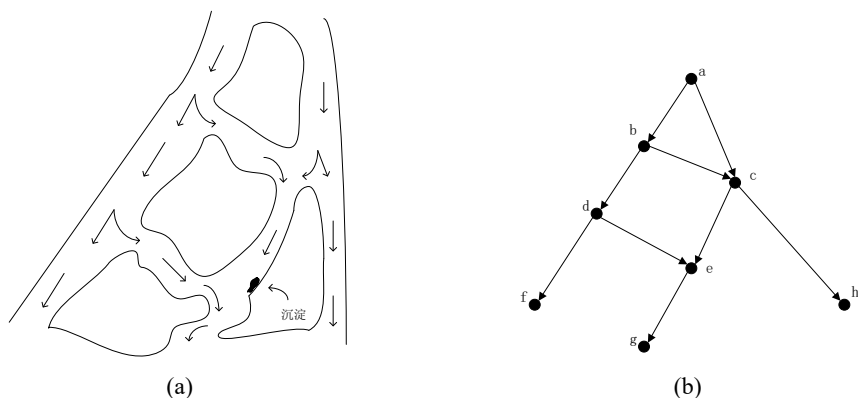


图 4.1 RFD 算法原理示意图

4.1.2 RFD 算法的实现过程

RFD 算法的实现方式包括以下步骤：

(1) 初始化水滴和图中节点。把水滴放置在初始位置，初始化图中所有节点的高度：把目标节点的高度设为 0 并在算法执行过程中保持不变，其余节点的高度设置为某个相等的值。

(2) 根据当前点与周围其他点的梯度计算不同路径的概率。在迭代过程中，当前点和下一目标点的梯度差越大，水滴越有可能流向这个目标点。水滴到达终点后会侵蚀它走过的路径。

(3) 执行第(2)步直到所有水滴都收敛到同一个解或者满足其他的结束条件。前一个条件要求所有水滴找到的路径都一样，后一个条件可以是满足一定的迭代次数、执行时间或者经过多次迭代后仍然没有找到更好的解。

(4) 最后一步分析水滴找到的解决方案，如果满足(3)中的条件就可以终止算法，否则继续进行迭代。

RFD 算法的流程如图 4.2 所示：

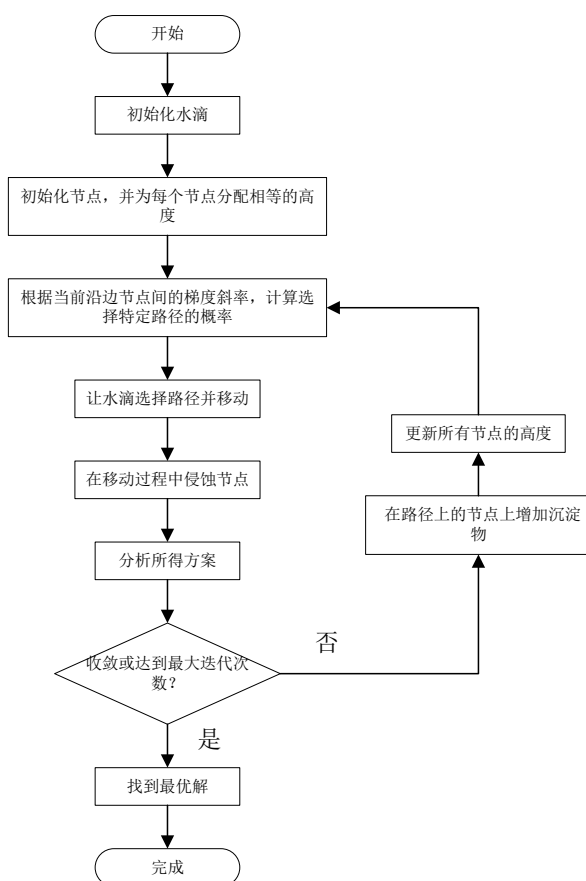


图 4.2 RFD 算法执行流程

水滴有可能在搜索过程中找不到解决方案。这种情况下，水滴将会被蒸发。所以每个水滴都有两种可能：

(1) 水滴找到了完整路径。这种情况下，水滴从起始节点移动到了目标节点，移

动过程中侵蚀所走路径。最终水滴返回起点重新开始搜索。

(2) 水滴蒸发。当水滴在搜索过程中周围所有相邻节点均被访问过，此时水滴不能找到解决方案，它所经过的路径也不会被侵蚀。

在初始阶段起始点周围都是平坦地形，对于这种情形需要给定一个概率可以保证水滴对平坦空间的深入探索；在后续的搜索过程中，水滴就会尽可能选择下降梯度更大的节点；搜索过程中还可能出现当前节点的高度低于周围节点的情况，为了避免出现这种情况也为了扩大搜索空间，水滴会有一个比较小的概率移动到更高的节点。综上所述，水滴 k 从节点 i 选择下一个节点 j 的概率转移规则为：

$$P_k(i, j) = \begin{cases} \frac{\text{gradient}(i, j)}{\text{sum} \cdot (d_j)^\alpha}, & \text{for } j \in V_k(i) \\ \frac{\omega / |\text{gradient}(i, j)|}{\text{sum} \cdot (d_j)^\alpha}, & \text{for } j \in U_k(i) \\ \frac{1}{P_{cl}}, & \text{for } j \in F_k(i) \end{cases} \quad (4-1)$$

其中， $V_k(i)$ 表示节点 i 具有正梯度的邻接点集合（节点 i 的高度大于节点 j ）， $U_k(i)$ 表示节点 i 具有负梯度的邻接点集合（节点 i 的高度小于节点 j ）， $F_k(i)$ 表示与节点 i 高度相等的邻接点集合。 d_j 表示下一可能节点 j 与目标节点（海洋）的距离；系数 ω 为常数； α 为收敛系数； sum 为节点 i 所有邻接点的梯度绝对值之和； P_{cl} 是一个可变参数，其初值设为 1，每经过几次循环它的值就会增加一次，类似于模拟退火算法的思想，到达一定值之后每经过 n 次迭代该变量的值开始减小。最重要的部分是两点间的梯度 $\text{gradient}(i, j)$ ，它由式 (4-2) 给出，

$$\text{gradient}(i, j) = \frac{\text{altitude}(i) - \text{altitude}(j)}{\text{distance}(i, j)} \quad (4-2)$$

其中 $\text{altitude}(i)$ 表示节点 i 的高度， $\text{distance}(i, j)$ 表示节点 i 和节点 j 的距离。

接下来会侵蚀前一阶段水滴的移动路径。如果水滴从节点 i 移动到节点 j ，那么节点 i 会被侵蚀。该节点降低的高度取决于两点之间的梯度：节点 i 和节点 j 的下降斜率越高，侵蚀效果就越强；如果是平坦或者上升斜率的路径，侵蚀效果会很弱。在这个过程中目标节点的高度始终为 0 不会发生变化。侵蚀过程后节点 i 的新高度定义如式 (4-3)，不同情况侵蚀方法如式 (4-4)：

$$\text{altitude}(i) = \text{altitude}(i) - \text{erosion}(i, j) \quad (4-3)$$

$$\text{erosion}(i, j) = \begin{cases} \frac{\varepsilon_V \cdot \text{gradient}(i, j)}{(N-1) \cdot M \cdot \text{pathLength}_k \cdot \varepsilon_U}, & \text{for } j \in V_k(i) \\ \frac{|\text{gradient}(i, j)| \cdot (N-1) \cdot M \cdot \text{pathLength}_k}{\varepsilon_F}, & \text{for } j \in U_k(i) \\ \frac{1}{(N-1) \cdot M \cdot \text{pathLength}_k}, & \text{for } j \in F_k(i) \end{cases} \quad (4-4)$$

其中, ε_V 、 ε_U 和 ε_F 是在不同梯度条件下的参数; $pathLength_k$ 表示水滴 k 已经走过的路径长度; N 和 M 分别表示图中节点数和水滴数。

为了避免出现所有节点高度都趋近于零导致梯度可以忽略不计、破坏所有路径的情况发生, 最后一步是向所有节点添加一定量的沉淀物, 使所有节点高度增加。在搜索到的最好路径上, 水滴会在每个节点上留下部分沉积物, 节点 i 增加后的新高度由下式给出:

$$altitude(i) = altitude(i) + \frac{erosionProduced}{N - 1} \quad (4-5)$$

其中, $erosionProduced$ 为前一阶段水滴侵蚀量的总和。

4.2 基于 ACO-RFD 的混合算法

4.2.1 蚁群算法和 RFD 算法的比较

蚁群算法是通过多只蚂蚁寻找从巢穴到食物源的不同路径, 从而找到最短路径的方法。蚂蚁在移动过程中会释放一些信息素, 这些信息素会把其他蚂蚁吸引到这条路径上, 路径信息也会共享给其他蚂蚁, 然后重复这条路径, 加强该路径上的信息素浓度。因为蚂蚁在单位时间内穿越短路径的次数要多于长路径, 所以信息素在短路径上的浓度比长路径更高。假设一只蚂蚁找到了一条新的去往食物源的路径, 而且这条路径比之前发现的都要短, 那么这条新路径就会逐渐吸引其他蚂蚁, 这些蚂蚁又会加强此路径, 经过多次循环迭代最终所有蚂蚁都沿着最短路径移动。蚁群算法通过在每个位置应用局部范围机制 (特别是信息素强化) 来构建良好的全局路径。

尽管蚁群算法可以通过信息素和正反馈的作用找到问题的解, 但它仍然存在一些内在的问题。一方面, 蚁群算法的思想在于通过信息素和蚂蚁合作的相互作用来构建良好路径。这个特点会带来双重效果, 路径彼此之间可能会相互干扰。例如, 蚂蚁可能找到只涉及某些节点的局部循环, 这个路径还可能会被随后的蚂蚁继续加强形成局部最优解。另一方面, 蚁群算法可能会遗漏一些图的简单特性, 如果把它们纳入考虑就能轻松找到更好的路径。比如, 当蚂蚁找到一条更好的路径时, 这条路径需要大量蚂蚁的加强才能使此路径的信息素浓度比之前最好路径的更高。

如果记录图中每个节点的信息素浓度, 当蚂蚁在决定下一步的移动位置时, 需要考虑两节点间信息素的差值, 保证差值为正 (即要求终点的信息素浓度高于起点) 才能选择这条路径。这样的设定可以在一定程度上避免上述两个问题。首先, 在搜索过程中不可能使所有边的信息素都是正增量, 因此不会形成循环反馈。关于路径的干扰大多数问题是由算法局部循环引起的。其次, 假设从节点 i 到节点 j 有不同路径可以到达, 如果一条路径被选取的概率与该路径的信息素浓度成正比, 与该路径的成本成反比, 那么蚂蚁会选择路径更短的那条。因为不同路径两点间信息素浓度的差值是相

等的,但较短的路径的成本显然更低。也就是说,新找到的较短路径的信息素在低于之前较好路径的时候,也不会出现许多蚂蚁被之前较好路径吸引的情况。

与蚁群算法相比,RFD 算法在这些方面更有优势。首先,它不会存在局部循环。尽管蚁群算法可以通过设置禁忌表记录已经访问过的节点来避免重复访问节点,但蚂蚁无法避免被信息素吸引到某些路径上,有可能出现走入死胡同的情况。在 RFD 算法中利用高度的思想不会出现这种情况,而且因为水滴不必关注后续的迭代过程,所以不需要记录访问信息,可以节省内存、减少运行时间。其次,RFD 算法发现更短路径后的强化速度很快,因为不管哪条路径,它们的起点和终点都是一样的,它们的高度差相同但距离不同。所以更短的路径必然具有更大的梯度,此路径会成为后续水滴的首选路径。相反,当蚂蚁找到了更好路径时,这条路径的信息素浓度与已找到的路径相比可以忽略不计,这条路径必须多次强化才能吸引更多蚂蚁。RFD 算法的侵蚀机制提供了一种高效率的解决方案:如果该点处在山谷中(它的高度小于所有相邻节点的高度),水滴仍能移动到下一点并在山谷处放置沉积物,慢慢地山谷会被填平。这与蚁群算法中蒸发信息素的方法不同,蒸发使得信息素周期性地减少,而 RFD 算法会惩罚坏路径、调整阻塞备选路径的节点。总的来说,蚁群算法能更快地得到可接受的解,但通常情况下 RFD 算法构造的解更好。

4.2.2 ACO-RFD 的混合算法

为了利用 ACO 和 RFD 的最佳特性,可以把两种算法结合起来(以下简称混合算法)。混合算法不单独使用各自的蚂蚁和水滴,而是设定一个新实体(称为混合实体),它包含了蚂蚁和水滴的全部属性,这也意味着混合实体会占用更多内存资源。当它寻找路径时,不仅会考虑边的信息素浓度还会考虑边的梯度。从节点 i 到节点 j 的概率 p_{ij} 计算方法为:首先按照蚁群算法的概率公式单独计算 p_{ij}^{ant} ,然后按照 RFD 算法的概率公式单独计算 p_{ij}^{drop} ,最后给两种概率分别赋予一个相对权重。设 ω_{ACO} 是 ACO 算法的权重, ω_{RFD} 是 RFD 算法的权重,保证 $\omega_{ACO} + \omega_{RFD} = 1$ 。最终混合实体的选择概率 $p_{ij} = \omega_{ACO}p_{ij}^{ant} + \omega_{RFD}p_{ij}^{drop}$ 。这两个相对权重在迭代过程中并不是固定不变的, ω_{ACO} 和 ω_{RFD} 会在迭代过程中动态变化同时保证二者之和为 1。算法在迭代开始时设定 $\omega_{RFD} = 1$,即使用 RFD 算法寻找初始解,然后这个值会慢慢减少到 0,再逐渐增加到 1。

对于双端布线问题,需要连接的两端可以是两个线网或者是两个引脚,其中一端称为源节点,另一端称为目的节点。在迭代开始时,混合实体会在源节点处进行初始化,每个混合实体也都从源节点出发寻找到达目的节点的路径。从一个点到另一个点的概率选择方式依然按照第三章的轮盘赌法,如果存在两个或多个节点的概率相等,就使用随机函数来选择其中一个节点。在找到路径后,记录每次找到的路径成本,在

迭代完成后进行比较。迭代完成后信息素浓度和节点高度按照各自方法独立更新，即沿所有边蒸发信息素，在路径上的每个节点都沉积沉淀物来增加节点高度。混合实体搜索到的最好路径会有信息素的增强、节点的侵蚀和沉淀物的沉积。这个过程不断重复直至找到最佳路线或达到最大迭代次数。

用混合算法解决双端布线的流程如图 4.3 所示。

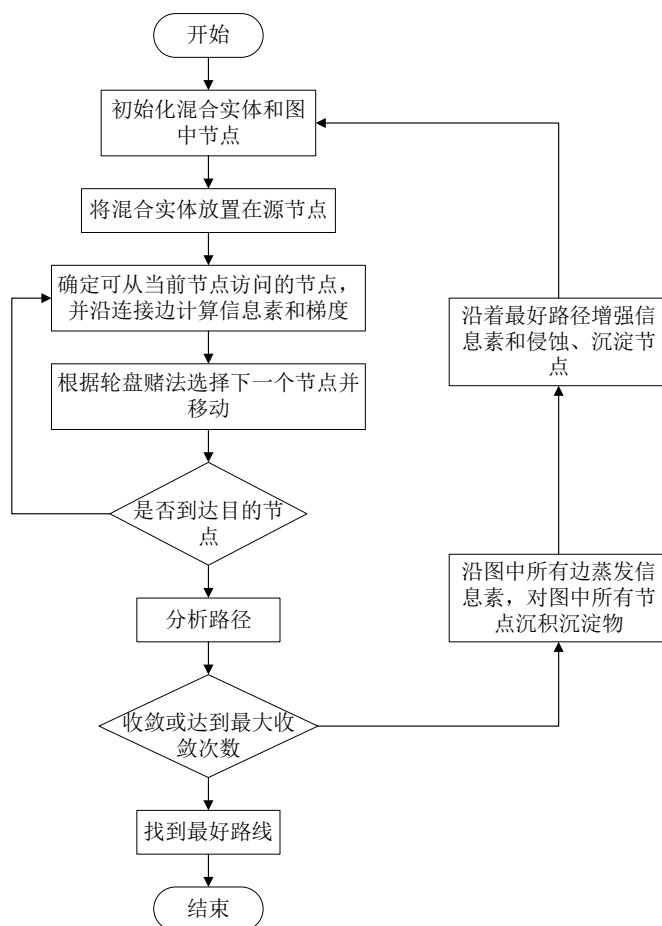


图 4.3 混合算法解决双端布线问题

4.3 基于 ACO-RFD 的混合算法的应用

4.3.1 多端、多组布线问题

除了双端布线问题外，还有多端布线和多组布线问题。多端布线问题需要连接两个以上的节点，它不能简单分解为多个双端布线的组合，多端布线的优化目标不再是两点间线路最优而是整个线网保证多点连接情况下的总线路最优。在曼哈顿架构下，多端线网布线可以转化为寻找该线网的 MRST 问题，此问题是一个 NP 完全问题。解决多端布线问题时可以在已布好的线路当中选中一点作为源节点，在未布端点集合中选择一点作为目的节点，多端布线的每一步都由双端布线来实现。在两个集合中选

择两点的要求是两点的成本函数是已布好的节点集合中所有点到未布节点集合中所有点之间的最小值。

多端布线的要求是从任意起点出发都能有路径到达所有端点，并不需要所有端点连接形成一个环路，所以从不同起点出发得到的路径可能会相差很大。对于图 4.4 的两种情况，这两种方式都能保证线网连通，但显然 (a) 图的路径长度更短。

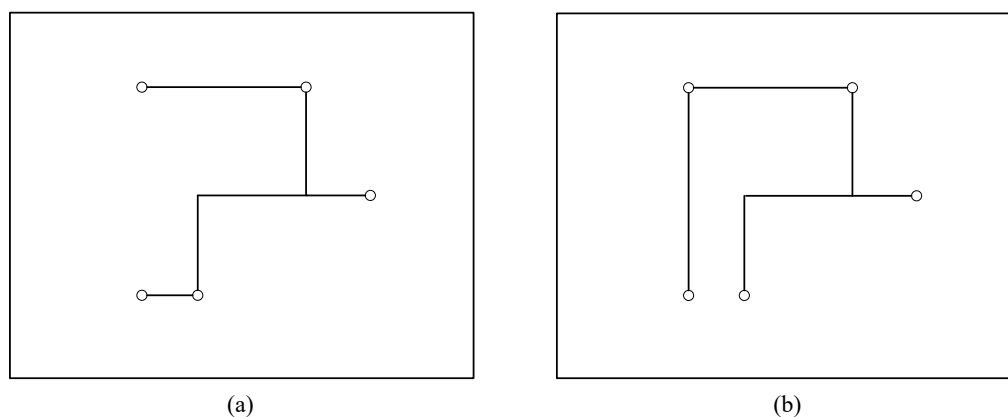


图 4.4 起点不同的多端布线

在多组布线问题中，线网的先后顺序是很重要的。各组线网布线的优先级顺序对布通率以及全局布线路径的影响作用是相当大的。如图 4.5 所示，如果先连接 3、4 两点的话，单层限制下 1、2 两点无论如何也不能连通；相反，如果先连接 1、2 两点，3、4 两点可以绕过 1、2 两点的路径连通。所以在布线过程中，可以动态调整线网的优先级实现布通率 100% 的要求。

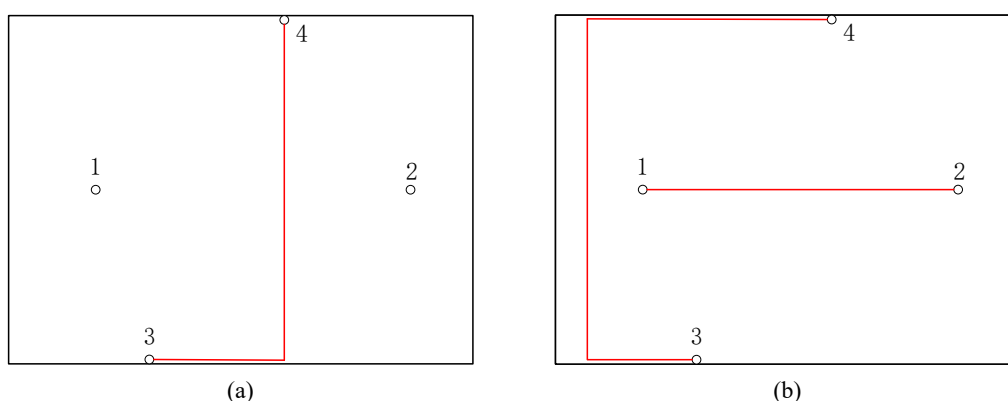


图 4.5 布线顺序的影响

4.3.2 加入线宽和线距约束

在真实的布线问题中，导线本身的宽度和导线间的距离也是不可忽视的。这些参数也会影响到电路的电阻、电容和电感等寄生参数，这些寄生参数会对电路特性产生

不良影响：增加传播延时、影响能耗和功率分布、引起额外的噪声来源，从而影响电路的可靠性。集成电路的线宽通常可理解为所加工的电路图形中最小线条宽度，集成电路线宽越小，代表其集成度和制造水平越高。加宽关键线网的连线宽度可以有效地提高时钟网的性能。导线宽度与其所能承载的电流大小也有关系，不同的线网可能导线宽度也不相同。除此之外，导线之间也会相互影响，若导线间距离过近，一条导线的电子可能发生量子隧穿效应而穿越到另一条导线中，这样很可能会影响电路的正确性和可靠性。

基于上述原因，在布线过程中不能把导线视作一根没有宽度的线段，要考虑到线宽和线距对布线空间的影响，这样导线所占据的布线空间就会更大。如图 4.6 所示，图中整个灰色部分表示导线线宽和线距之和，即为一根导线占据的布线区域。

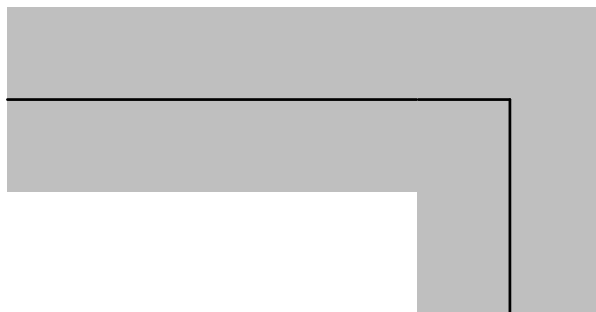


图 4.6 导线的线宽和线距

对于不同类型的线网，线宽和线距的要求可能会不同。在双端布线问题中，因为仅有两个端点，只需要一次布线即可完成，所以线宽和线距并不会影响布线过程；在多端布线中，有些端点布线前布线空间已经存在一部分导线，这些导线的存在会挤占布线空间，从而影响后续的布线工作；多组布线不仅会出现多端布线的问题，每一组线网也都会占用更大的布线空间，对其他组的布线造成困难。

4.3.3 布线问题的解决

(1) 双端布线

双端布线是最为简单的布线问题，多端布线和多组布线都是在它的基础上进行的。图 4.7 为双端布线的仿真结果，布线空间为 100×100 ，图中虚线框表示障碍，布线空间中大小不一的矩形障碍共有 60 个。

图 4.8 为蚁群算法、RFD 算法和混合算法在双端布线的迭代过程，三种算法中蚁群算法收敛速度相对较快，混合算法收敛速度虽然慢于蚁群算法，但其搜索到的解的质量更好。在后续布线过程中，算法在收敛于某个值达到一定次数后，搜索空间会被重新初始化。这样在同样的迭代次数下，能够多找到几组解进行比较，找到的最优解的质量可能更好。

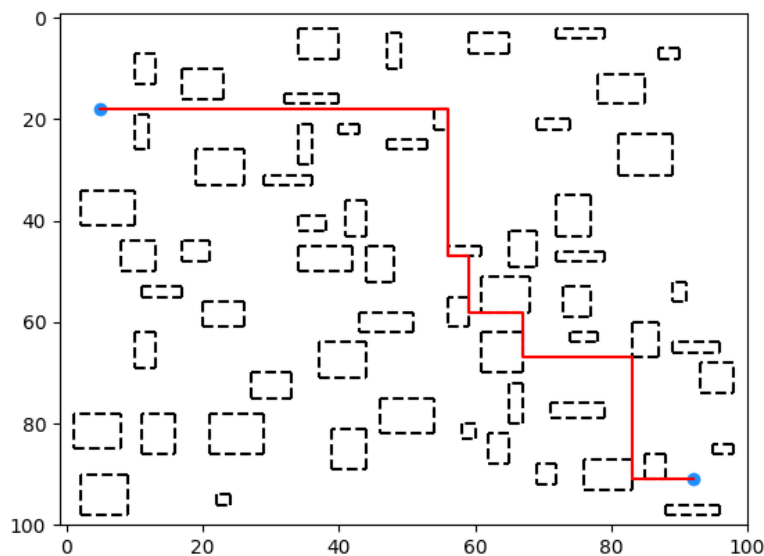


图 4.7 双端布线

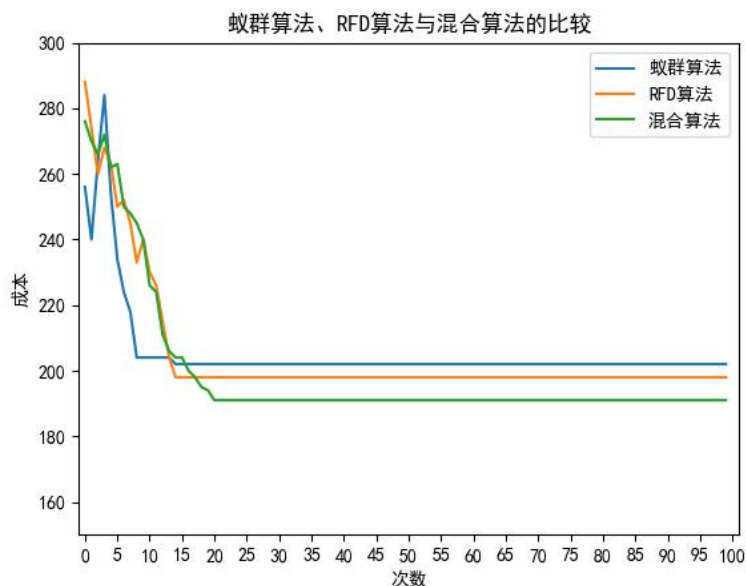


图 4.8 三种算法的收敛情况

表 4.1 给出蚁群算法、RFD 算法和混合算法在 5 种不同布线条件下的表现。对每个测试用例分别运行 30 次，得到三种算法找到的最优解，并求出 30 次解的平均值。从表中可以看出，在布线规模很小而且障碍数很少的情况下，布线空间比较充足，布线相对比较灵活，三种算法都能够找到全局最优解，平均值相差不大。图 4.9 以第 5 组数据为代表展示了三种算法 30 次的迭代结果，总体来看混合算法效果最好，RFD 算法次之，蚁群算法效果最差且不稳定。图 4.10 展示了 5 组用例不同算法的最优解和平均值的对比，布线规模较小的情况下，三者差距很小，但随着问题变得更复杂，三者之间的差异也更加明显，混合算法得到的解大概率都优于蚁群算法和 RFD 算法的解。

表 4.1 双端布线实验数据

用例	布线规模		蚁群算法		RFD 算法		混合算法	
	大小	障碍数	最优值	平均值	最优值	平均值	最优值	平均值
1	30*30	20	50	55.81	50	55.85	50	55.93
2	100*100	100	94	96.83	91	96.18	89	95.33
3	100*100	225	90	106.87	86	103.63	78	101.1
4	150*150	500	174	216.4	166	210.23	156	201.93
5	200*200	900	367	451.93	343	423.53	320	400.87

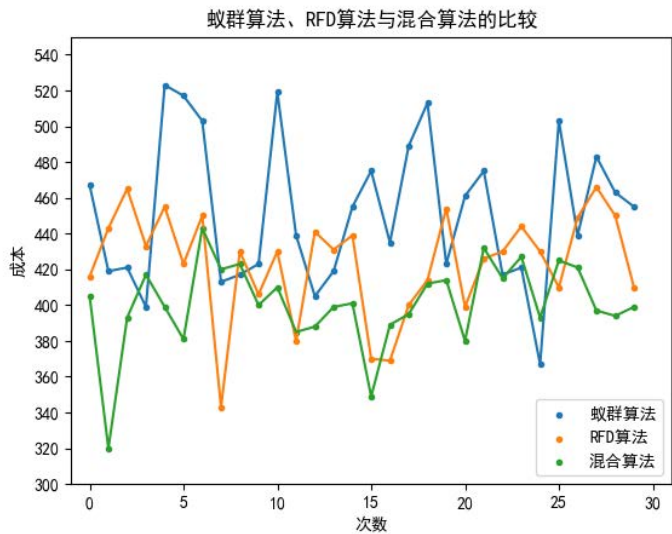


图 4.9 双端布线的迭代过程

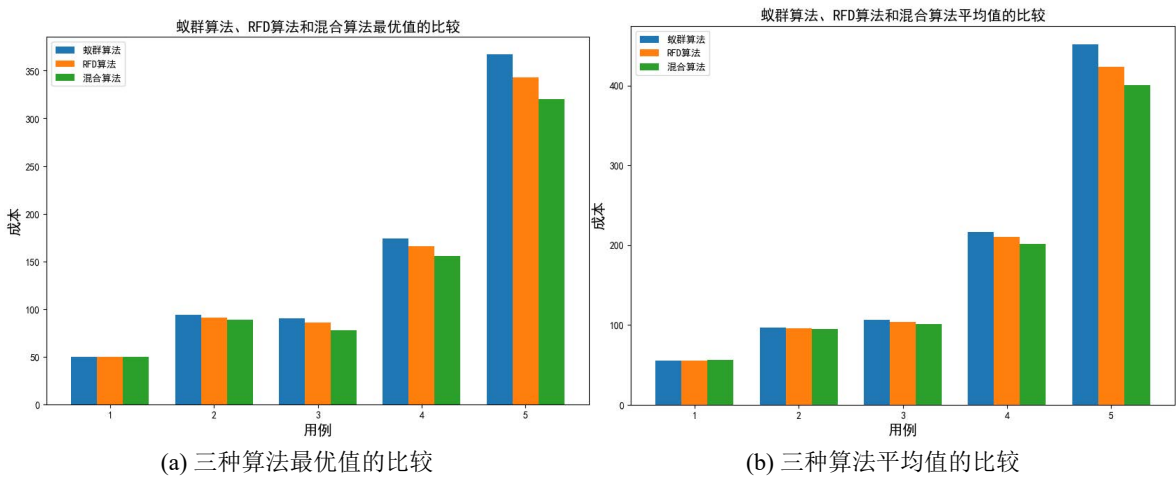


图 4.10 双端布线最优值和平均值的对比

(2) 多端布线

多端布线不是由多个双端布线问题简单组合而成的，多端布线要求所有端点连接起来，而不用必须从一个点出发到另一个点，也不需要形成一个环。正因为如此，

以不同端点作为源节点，所得到的答案差距可能是很大的。在搜索过程中，可以从不同端点出发，比较不同路径找出最优解。当布线空间已经存在导线时，其线宽和线距占据的空间会影响到其他端点的连接过程。图 4.11 为多端布线的仿真结果，布线空间为 100×100 ，有 60 个障碍和 6 个端点，线宽和线距设置为 2。

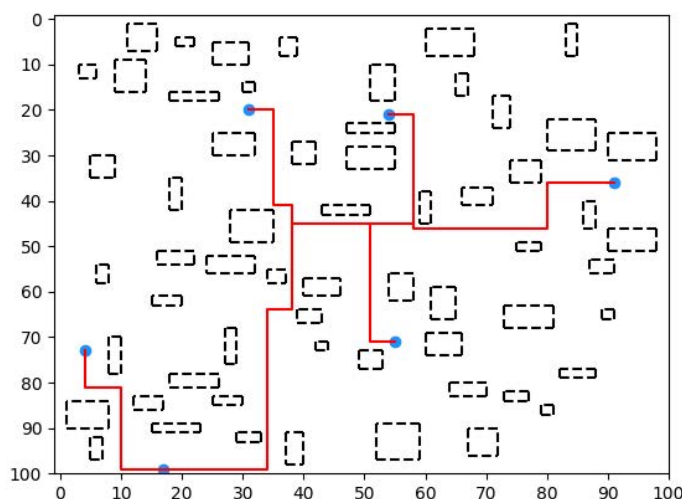


图 4.11 多端布线

展示的仿真数据同双端布线情况，表 4.2 给出蚁群算法、RFD 算法和混合算法在 8 种不同情况下的布线结果。多端布线需要连接的端点数量越多，布线情况就越复杂，已经布好的路径对布线空间的影响更大，它不以两点之间的路径成本最优为目标而是要保证整条线路成本最优。图 4.12 以第 3 组实验数据为例，展示了三种算法的迭代过程。图 4.13 为三种算法在不同用例下最优值和平均值的比较，在布线空间大小和障碍数相同的情况下，端点数越多，三种算法的差异就越大，更能体现出混合算法的寻优能力。

表 4.2 多端布线实验数据

用例	布线规模			蚁群算法		RFD 算法		混合算法	
	大小	障碍数	端点数	最优值	平均值	最优值	平均值	最优值	平均值
1	100*100	100	10	318	354.97	317	354.13	315	352.90
2	100*100	100	20	390	423.30	375	416.37	366	410.67
3	100*100	225	10	312	333.60	307	321.26	296	314.32
4	100*100	225	20	456	516.00	436	478.34	419	458.90
5	150*150	500	20	606	652.87	589	643.28	580	637.80
6	150*150	500	30	826	879.97	811	856.41	797	845.60
7	200*200	900	30	1196	1279.26	1163	1213.63	1113	1183.26
8	200*200	900	40	1634	1756.47	1598	1723.19	1553	1674.46

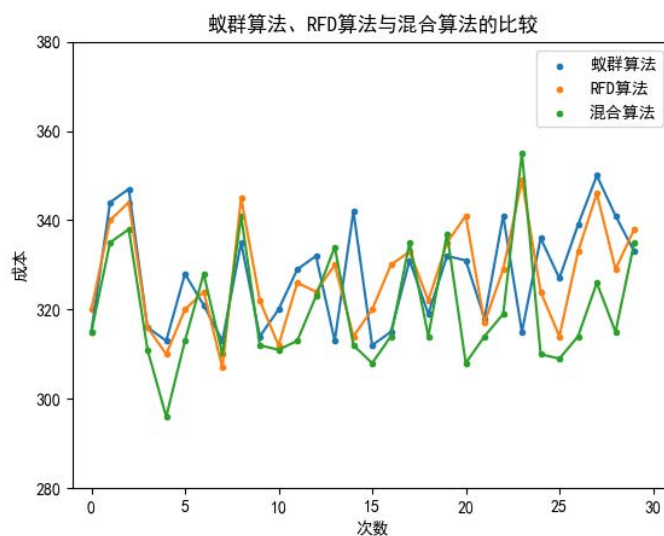


图 4.12 多端布线迭代过程

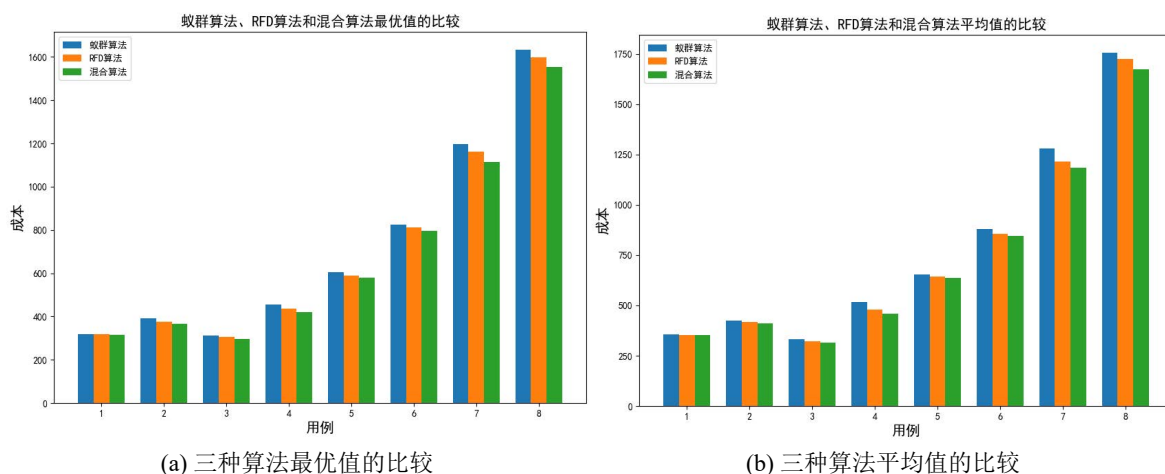


图 4.13 多端布线最优值和平均值的对比

(3) 多组布线

在多组布线中, 每组线网端点数目可能是两个也可能是多个, 根据每组线网端点数目的不同, 可以用双端布线或者多端布线实现。多组布线更加需要注意的是组与组之间的影响, 每个线网的优先级不同, 最后布线路径的质量就不同, 甚至可能会出现不能布通的情况。图 4.14 为多组多端布线的仿真结果, 布线空间为 100×100 , 有 80 个障碍, 3 组线网, 线宽和线距设置为 2。

多组布线中, 线网数量和每个线网的复杂程度都会影响到布线问题的难易。表 4.3 为蚁群算法、RFD 算法和混合算法在 8 种不同情况下的布线结果。图 4.15 展示了第 5 组示例的蚁群算法、RFD 算法和混合算法的迭代结果。图 4.16 展示了三种算法得到的最优值和平均值的差异, 总体上线网数目越多, 算法效果差异越显著。

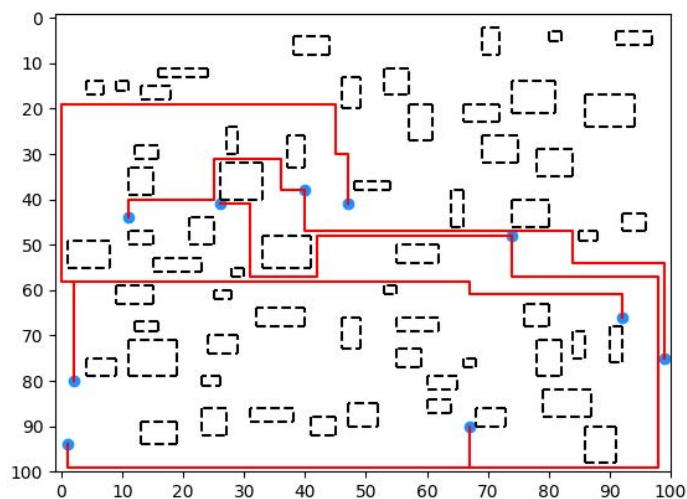


图 4.14 多组布线

表 4.3 多组布线实验数据

用例	布线规模			蚁群算法		RFD 算法		混合算法	
	大小	障碍数	线网数	最优值	平均值	最优值	平均值	最优值	平均值
1	100*100	100	3	464	508.03	450	492.32	444	488.20
2	100*100	100	6	339	368.63	336	365.13	328	355.73
3	100*100	225	3	351	395.43	323	351.16	318	345.67
4	100*100	225	6	360	401.35	344	381.76	321	355.46
5	150*150	500	10	592	621.97	588	618.73	580	609.80
6	150*150	500	15	667	723.15	645	681.69	631	660.14
7	200*200	900	15	1013	1079.30	989	1043.40	950	973.17
8	200*200	900	20	1452	1513.54	1410	1466.17	1354	1391.42

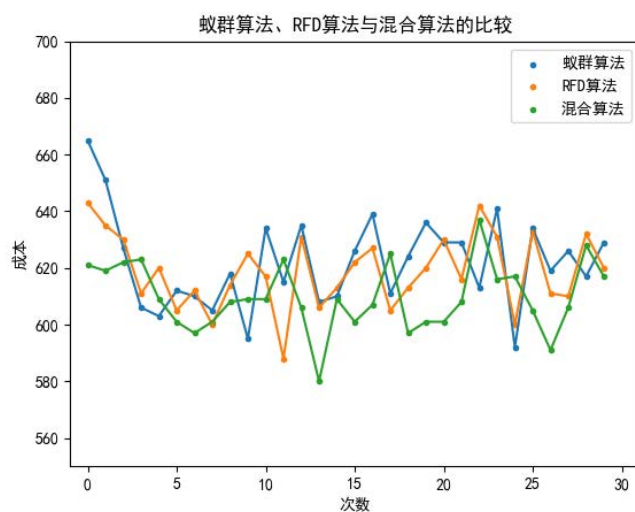


图 4.15 多组布线迭代过程

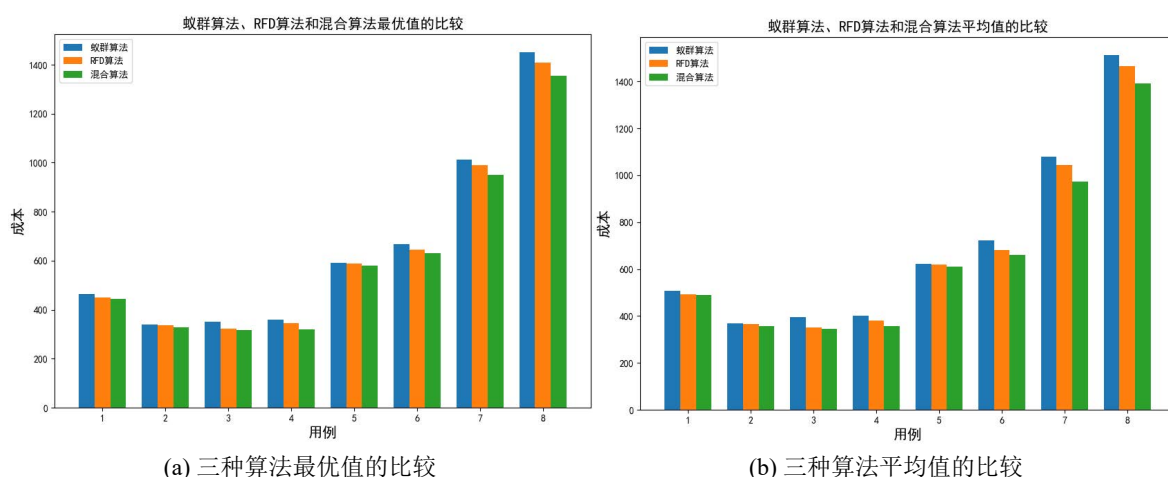


图 4.16 多组布线最优值和平均值的对比

4.4 多层布线

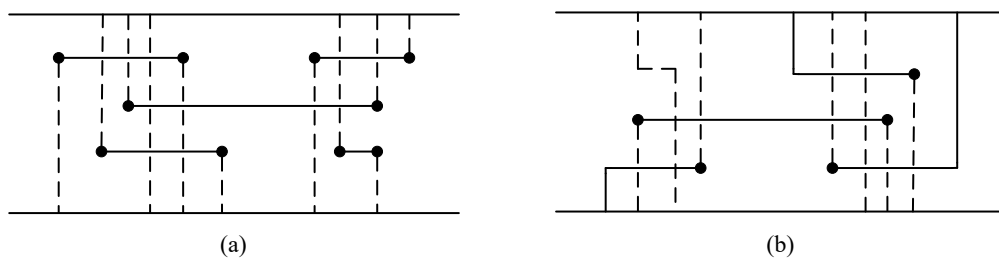
对于一些特别复杂的线网，仅在一个平面上进行布线是不可能达到 100% 布通率的，这时就需要加入更多的布线层来拓展布线空间。层与层之间通过硅通孔（Through Silicon Via, TSV）进行连接，这样器件可以分布在不同层中，互连线可以通过 TSV 穿越各层。

在双层布线模型中，选择一层进行水平布线，另一层进行垂直布线。不同层之间使用通孔进行连接，从技术角度看，通孔本身的电阻会影响电路性能，其尺寸大于导线线宽也会使布线空间的需求增大，所以通孔数量应该保持很少。然而大多数双层布线算法会产生大量通孔。参考双层布线，多层布线算法中每一层也只能选择在一个方向上（垂直或水平）布线，垂直层和水平层交替出现，但是仍然会产生许多通孔。在集成电路中如果通孔数量过多不仅会降低芯片性能和成品率，还会增加制造成本和芯片尺寸。因此在布线算法中必须考虑最小化通孔的数量。

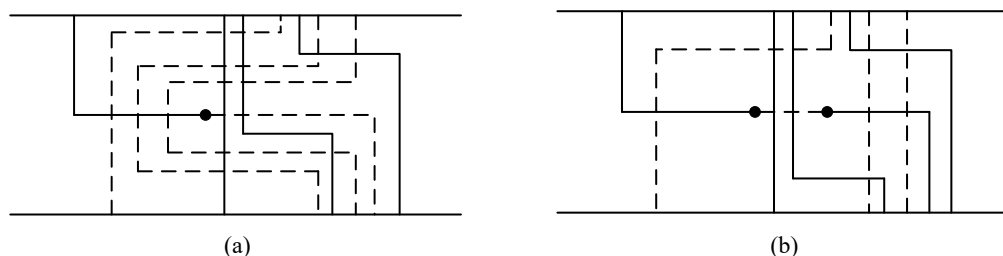
通孔最小化问题可以通过决定布线网络的拓扑结构和布线路径的层分配来解决。通孔最小化问题可以分为两大类：

约束通孔最小化（Constrained Via Minimization, CVM）。在完成详细布线后，通过把连线分配到不同的层来减少通孔数量。CVM 布线方案的拓扑结构没有发生变化，如图 4.17 所示，图 4.17(a) 表示通孔最小化之前的布线情况，布线需要 10 个通孔，图 4.17(b) 中，在保持相同布局的条件下只使用 5 个通孔就完成布线。

无约束通孔最小化（Unconstrained Via Minimization, UVM）。在通孔最小化中，如果对完成的布线方案没有任何其他限制，则被称为无约束最小化。在 UVM 中，布线的实际布局可以被改变，如图 4.18 所示。虽然 UVM 在大多数情况中可以减少通孔数量，但布局通常不紧凑，会占用更大的面积。在图 4.18(a) 中，实现最小化通孔

图 4.17 约束通孔最小化^[85]

数量需要线网在通孔周围环绕，在几何映射中，环绕的线网会使用两根或多根导线。与图 4.18(b) 相比，如果再增加一个通孔，需要更少的导线就可以完成布线。

图 4.18 无约束通孔最小化^[85]

实现 CVM 还可以通过使用最小层数消除边交叉，达到减少通孔的目的。首先可以先忽略导线不能交叉的限制，找到各个线网的最优路径，这样得到的是多层布线结果在单层上的投影，投影图是存在交叉的布线结果，存在交叉代表需要通孔连接。把交叉的连线分配的不同层中，从而实现移动最小数量的连线同时保持最小数量的层数。

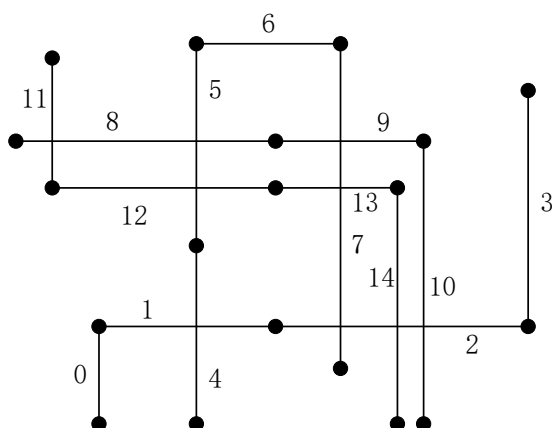


图 4.19 多层布线初始结果

以图 4.19 为例，图中展示了 4 组多端线网最优路径的初始结果，把每两个节点之间的连线叫做边。构造出一个具有 N 个顶点的交叉图，如图 4.20 所示，其中每个

顶点表示图 4.19 中的一条边。如果边 e_i 和边 e_j 之间存在交叉，就在顶点 v_i 和 v_j 之间画一条连线。之后计算每条边的交叉情况，构造一个 $N \times (N + 1)$ 的矩阵 M （多的一列表示图中节点的度），如表 4.4 所示。其值 $M_{ij} = M_{ji} = 1$ 表示第 i 条边和第 j 条边相交，值 $M_{ij} = M_{ji} = 0$ 则表示两边不相交。

表 4.4 矩阵 M

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	度
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1
2	0	0	0	0	0	0	0	1	0	0	1	0	0	0	1	3
3	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
4	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
5	0	0	0	0	0	0	0	0	1	0	0	0	1	0	0	2
6	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
7	0	0	1	0	0	0	0	0	0	1	0	0	0	1	0	3
8	0	0	0	0	0	1	0	0	0	0	0	1	0	0	0	2
9	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	1
10	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1
11	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	1
12	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1
13	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	1
14	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1

边与边存在交叉点就意味着需要通孔进行连接，为了移除交叉点，可以把一些边转移到不同层，使每个层都没有交叉点。首先在矩阵中找到度最大的节点，如果能找到多个值相同的节点就选择第一个节点，记作节点 i ，并使 $M_{ij} = M_{ji} = 0$ ，构造集合 $S = \{S + e_i\}$ 。其中 S 初始时空集，它记录了需要从当前层移至下一层的边的集合。然后更新每个节点的度，再次检查是否有度非零的节点，如果有的话就重复上述过程直至当前层所有节点的度都为 0。后续再按照上述步骤计算下一层的顶点交叉情况，以此类推直至最后一层没有交叉点。

根据上述过程找到了需要被移除的边，即图 4.20 的顶点 1、2、5、7、11，该集合包含最少的节点，使交叉图中的顶点都与该集合中的至少一个顶点相关联。因为每个顶点代表了一条边，所以该集合就是使图 4.19 不交叉而移除掉的最小边集。

把最小边集中的边移动至第二层中，得到图 4.21，第二层的边用虚线表示，检查第二层的边是否有交叉点，发现仍有边交叉，把第 2 条边移动至第三层，算法结束。

该多层布线算法描述如下：

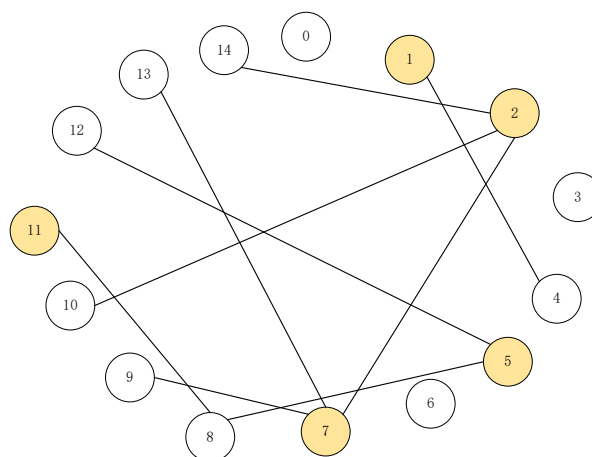


图 4.20 顶点交叉图

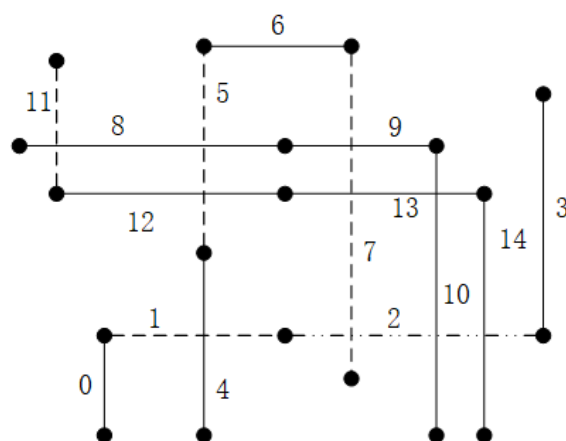


图 4.21 多层布线最终结果

算法 4.1 多层布线算法

输入: G : 布线图

E : 图 G 的边集, $E = \{e_i | i = 0, 1, 2, \dots, N - 1\}$

输出: L : 边的分层结果

变量: S : 需要移动到下一层的边集, 初始化为空集

N : 图 G 的边数

D : 图 G 每个顶点的度

p : 数组 D 中最大值的索引

```

1: for  $i \leftarrow 0, N - 1$  do
2:   for  $j \leftarrow 0, N - 1$  do
3:      $M_{ij} \leftarrow 0$ 
4:   end for
5: end for
    
```

```

6: for  $i \leftarrow 0, N - 1$  do
7:    $D_i \leftarrow 0$ 
8: end for
9: for  $i \leftarrow 0, N - 1$  do
10:   for  $j \leftarrow 0, N - 1, j \neq i$  do
11:     if  $e_i$  与  $e_j$  相交 then
12:        $M_{ij} \leftarrow 1$ 
13:     end if
14:   end for
15: end for
16: for  $i \leftarrow 0, N - 1$  do
17:   for  $j \leftarrow 0, N - 1$  do
18:     if  $M_{ij} = 1$  then
19:        $D_i \leftarrow D_i + 1$ 
20:     end if
21:   end for
22: end for
23:  $p \leftarrow 0$ 
24: for  $i \leftarrow 1, N - 1$  do
25:   if  $D_p < D_i$  then
26:      $p \leftarrow i$ 
27:   end if
28: end for
29:  $S \leftarrow S + e_p$ 
30: for  $i \leftarrow 0, N - 1$  do
31:    $M_{pj} \leftarrow 1, M_{jp} \leftarrow 1, D_p \leftarrow 1$ 
32: end for
33: for  $i \leftarrow 0, N - 1$  do
34:   if  $D_i \neq 0$  then
35:     goto line10
36:   else
37:     结束
38:   end if
39: end for

```

该算法在最坏情况下的时间复杂度为 $O(N^3)$ ，随着交叉点数量的增多，算法运行时间就更长。随机生成若干组多端线网，得到的结果如下表所示。

表 4.5 多层布线实验结果

用例	大小	线网数	路径长度	交叉点个数	层数
1	50*50	3	61	3	2
2	50*50	5	118	14	3
3	100*100	5	432	36	3
4	100*100	7	549	51	4
5	150*150	7	694	24	3
6	150*150	9	731	53	4
7	200*200	9	1024	37	4
8	200*200	11	1284	49	5

多层布线问题需要的层数主要取决于线网的复杂程度，如果某个区域的拥挤度很高，许多连线在此区域产生大量交叉的话，边之间的关系会十分复杂，需要分配更多的布线层才能将边与边不交叉。

4.5 本章小结

本章首先介绍了 RFD 算法的原理和实现过程，将 RFD 算法与蚁群算法相结合组成混合算法，该混合算法在蚁群算法的基础上增加了 RFD 算法的参数和机制，更新了选择概率公式。这样弥补了蚁群算法的不足之处，提高了算法的寻优能力。之后将蚁群算法、RFD 算法和混合算法分别应用到双端布线、多端布线和多组布线问题上，通过仿真结果可以验证混合算法要优于蚁群算法和 RFD 算法。最后在多层布线问题中，通过将连线分布到尽可能少的层来解决通孔最小化问题。

第五章 总结与展望

5.1 总结

随着集成电路的特征尺寸越来越接近物理极限，互连线的延时和功耗问题愈发突出，良好的布线方案不仅可以解决时钟同步问题，也可以降低电路延时、功耗并且能减少芯片面积。以目前集成电路的复杂程度，布线问题是不可能通过人工来完成的，所以需要通过使用智能算法由计算机完成布线工作。集成电路规模的扩大使得布线在单层上无法有效实现，多层布线可以拓展布线空间，层之间通过通孔连接，但要通过一定方式解决通孔最小化问题。本文主要工作内容如下：

第一章为绪论部分，介绍了课题背景和研究意义、集成电路的发展过程和未来的发展趋势以及 EDA 技术发展历史。叙述了国内外对蚁群算法和集成电路布线问题的研究现状。

第二章介绍了集成电路设计阶段的各个流程，其中物理设计在整个流程中十分关键，起到了承上启下的作用。着重介绍了物理设计的电路划分、布局规划、布线和压缩。本文研究重点主要在布线阶段，因此对于全局布线和详细布线展开说明。

第三章引入了蚁群算法，以 TSP 为背景对其算法原理、实现过程进行了详细描述，针对蚁群算法的选择概率公式和信息素更新策略进行优化，使算法的寻优能力得到提升。将蚁群算法应用到布线问题中首先要构建布线模型，本文使用曼哈顿架构的无网格布线模型，通过扫描形成的水平和垂直轨线来构建路径图。最后提出了布线问题中的优化目标，使用轮盘赌选择机制，并在有网格模型和无网格模型上分别布线，比较了二者的差异。

第四章介绍了 RFD 算法的原理和实现过程，分析蚁群算法存在的不足，将蚁群算法和 RFD 算法结合来提升算法性能。后续介绍了双端布线、多端布线和多组布线问题，引入线宽和线距的约束使布线更贴近实际。然后将蚁群算法、RFD 算法和混合算法应用到不同情况的双端布线、多端布线和多组布线当中，验证了改进后算法的优越性。最后介绍了多层布线中通孔最小化的两种方式，给出多层布线的算法流程和仿真结果，通过实验展示了算法的可行性。

5.2 展望

蚁群算法和 RFD 算法都具有并行思想，很适用于并行执行，算法可以在图形处理单元（Graphics Processing Unit）上运行，应用统一计算设备框架（Compute Unified Device Architecture）开发的很多程序相较于在中央处理器（Central Processing Unit）运

行的程序有数十倍的加速比，从而可以减少算法的运行时间。在进行多组布线时，每一组线网都可以同时布线而不用考虑每组布线顺序问题，但并行布线的策略会更加复杂，实现难度较大，后续在这方面可以深入研究。

RFD 算法相较于蚁群算法提出时间较晚，在研究深度和应用领域范围都不如蚁群算法，而且 RFD 算法涉及的参数更多，很多情况下选取相关参数靠的是经验和实验，没有一组参数选择方法或通用参数可供参考。需要在后面的研究中找到更好的选取参数的方式，扩大算法的应用领域。

本文的多组布线是通过调整不同组别的优先级来完成布通目标的，在有些情况下可以保持优先级不变，调整布线路径也可以实现该目标。这就需要对算法知道对于不同情况应该如何处置，现在人工智能技术发展迅速，今后可以将蚁群算法与人工智能结合起来完成布线工作。

从 EDA 领域的专家口中得知，在实际的集成电路设计中只通过一种算法不能完成布线工作，针对不同情况会选择适合的算法进行布线。后续研究可以注重不同算法在不同场景下的效果。

本文用的布线模型是基于曼哈顿架构的，近些年关于 X 架构和 Y 架构的研究较多，应用也比较广泛，今后可以研究在不同的布线架构下布线的差异。

参考文献

- [1] 王小强, 邓传锦, 范剑峰. 集成电路发展历程、现状和建议 [J]. 电子产品可靠性与环境试验, 2021, 39(S1): 106-111.
- [2] WALDROP M M. More than moore[J]. Nature, 2016, 530(7589): 144-148.
- [3] TRACK E, FORBES N, STRAWN G. The end of moore's law[J]. Computing in Science & Engineering, 2017, 19(2): 4-6.
- [4] CHEN J Y. Transformation of VLSI technologies, systems and applications the rise of foundry and its ecosystem[C]//2013 International Symposium on VLSI Technology, Systems and Application (VLSI-TSA). IEEE, 2013: 1-2.
- [5] 滕冉. “十四五”期间我国集成电路行业发展趋势特征分析 [J]. 中国集成电路, 2021, 30(09): 26-28.
- [6] 朱晶, 赵佳菲, 史弘琳, 等. 2020 年中国集成电路产业现状回顾和新时期发展展望 [J]. 中国集成电路, 2020, 29(11): 11-16+19.
- [7] REIS R. Trends on EDA for low power[C]//2015 IEEE MTT-S International Conference on Numerical Electromagnetic and Multiphysics Modeling and Optimization (NEMO). IEEE, 2015: 1-4.
- [8] 李东生. 电子设计自动化与 IC 设计 [M]. 高等教育出版社, 2004.
- [9] 刘超, 王珺, 张天仪. 集成电路 EDA 行业发展态势分析 [J]. 中国集成电路, 2021, 30(06): 32-37.
- [10] 刘星, 夏璠. 数字集成电路 EDA 软件行业发展前景探析 [J]. 中国信息化, 2021(09): 117-120.
- [11] COLORNI A, DORIGO M, MANIEZZO V, et al. Distributed optimization by ant colonies[C]//Proceedings of the First European Conference on Artificial Life: volume 142. Paris, France, 1991: 134-142.
- [12] BLUM C, VALL E S M Y A B, BLES A M J. An ant colony optimization algorithm for DNA sequencing by hybridization[J]. Computers & Operations Research, 2008, 35(11): 3620-3635.
- [13] LI W, WANG S, CHAKRADHAR S T, et al. Distance restricted scan chain reordering to enhance delay fault coverage[C]//18th International Conference on VLSI Design held jointly with 4th International Conference on Embedded Systems Design. IEEE, 2005: 471-478.
- [14] BLUM C. Beam-aco—hybridizing ant colony optimization with beam search: An application to open shop scheduling[J]. Computers & Operations Research, 2005, 32(6): 1565-1591.
- [15] GAMBARDILLA L M, TAILLARD E, DORIGO M. Ant colonies for the QAP[R]. Citeseer, 1997.
- [16] KORB O, ST U TZLE T, EXNER T E. An ant colony optimization approach to flexible protein—ligand docking[J]. Swarm Intelligence, 2007, 1(2): 115-134.
- [17] BLUM C. Beam-aco for simple assembly line balancing[J]. INFORMS Journal on Computing,

- 2008, 20(4): 618-627.
- [18] 周森鹏, 穆平安, 张仁杰. 基于烟花混合蚁群的移动机器人路径规划研究 [J]. 包装工程, 2019, 40(11): 172-176.
- [19] 王雷, 石鑫. 基于改进蚁群算法的移动机器人动态路径规划 [J]. 南京理工大学学报, 2019, 43(6): 700-707.
- [20] LIL, YANG Y, PENG H, et al. Parameters identification of chaotic systems via chaotic ant swarm[J]. Chaos, Solitons & Fractals, 2006, 28(5): 1204-1211.
- [21] 毛晓明, 蔡永智, 赵勇. 采用最大-最小蚁群算法的励磁系统参数辨识 [J]. 电力系统及其自动化学报, 2015, 27(05): 51-55.
- [22] SHUNMUGAPRIYA P, KANMANI S. A hybrid algorithm using ant and bee colony optimization for feature selection and classification (AC-ABC Hybrid)[J]. Swarm and Evolutionary Computation, 2017, 36: 27-36.
- [23] 何小虎. 基于优化的蚁群图像边缘检测算法研究 [J]. 计算机技术与发展, 2017, 27(02): 60-63.
- [24] 林树锋, 王冬姣, 叶家玮, 等. 基于蚁群算法的水下机器人机械臂工作路径优化 [J]. 水下无人系统学报, 2019, 27(01): 45-50.
- [25] 田云娜, 李冬妮, 郑丹, 等. 一种基于时间窗的多阶段混合流水车间调度方法 [J]. 机械工程学报, 2016, 52(16): 185-196.
- [26] 焦留成, 邵创创, 程志平. 一种求解连续空间约束优化问题的蚁群算法 [J]. 郑州大学学报 (工学版), 2015, 36(01): 20-23.
- [27] MAVROVOUNIOTIS M, YANG S. Ant algorithms with immigrants schemes for the dynamic vehicle routing problem[J]. Information Sciences, 2015, 294: 456-477.
- [28] RAMKUMAR A S, PONNAMBALAM S G, JAWAHAR N. A population-based hybrid ant system for quadratic assignment formulations in facility layout design[J]. The International Journal of Advanced Manufacturing Technology, 2009, 44(5): 548-558.
- [29] 毛嘉琪. 改进蚁群算法的机器人路径规划研究 [J]. 计算机应用与软件, 2021, 38(05): 300-306.
- [30] 任红格, 胡鸿长, 史涛. 基于改进蚁群算法的移动机器人动态路径规划 [J]. 现代电子技术, 2021, 44(20): 182-186.
- [31] ISMKHAN H. Effective heuristics for ant colony optimization to handle large-scale problems[J]. Swarm and Evolutionary Computation, 2017, 32: 140-149.
- [32] LUO Q, WANG H, ZHENG Y, et al. Research on path planning of mobile robot based on improved ant colony algorithm[J]. Neural Computing and Applications, 2020, 32(6): 1555-1566.
- [33] YU B, YANG Z Z, YAO B. An improved ant colony optimization for vehicle routing problem[J]. European Journal of Operational Research, 2009, 196(1): 171-176.
- [34] 张天瑞, 吴宝库, 周福强. 面向机器人全局路径规划的改进蚁群算法研究 [J]. 计算机工程与应

- 用, 2022, 58(1): 282-291.
- [35] ZHANG Y, PEI Z L, YANG J H, et al. An improved ant colony optimization algorithm based on route optimization and its applications in travelling salesman problem[C]//2007 IEEE 7th International Symposium on BioInformatics and BioEngineering. IEEE, 2007: 693-698.
- [36] DENG W, XU J, ZHAO H. An improved ant colony optimization algorithm based on hybrid strategies for scheduling problem[J]. IEEE Access, 2019, 7: 20281-20292.
- [37] ESCARIO J B, JIMENEZ J F, GIRON-SIERRA J M. Ant colony extended: experiments on the travelling salesman problem[J]. Expert Systems with Applications, 2015, 42(1): 390-410.
- [38] 董晨. 粒子群优化的超大规模集成电路全局布线策略 [D]. 武汉大学, 2011.
- [39] JOSE D, NIRMAL KUMAR P, HUSSAIN A, et al. VLSI circuit partitioning using ant colony optimisation to yield fault tolerant testable systems[J]. Arabian Journal for Science and Engineering, 2014, 39(12): 8709-8729.
- [40] PEYER S, RAUTENBACH D, VYGEN J. A generalization of Dijkstra's shortest path algorithm with applications to VLSI routing[J]. Journal of Discrete Algorithms, 2009, 7(4): 377-390.
- [41] YANG Z, VANNELLI A, AREIBI S. An ILP based hierarchical global routing approach for VLSI ASIC design[J]. Optimization Letters, 2007, 1(3): 281-297.
- [42] DENG X, YAO Y, CHEN J, et al. Combining breadth-first with depth-first search algorithms for VLSI wire routing[C]//2010 3rd International Conference on Advanced Computer Theory and Engineering (ICACTE): volume 6. IEEE, 2010: V6-482.
- [43] MANNA S, CHAKRABARTI T, SHARMA U, et al. Efficient VLSI routing optimization employing discrete differential evolution technique[C]//2015 IEEE 2nd International Conference on Recent Trends in Information Systems (ReTIS). IEEE, 2015: 461-464.
- [44] AYOB M N, HASSAN F, ISMAIL A H, et al. A firefly algorithm approach for routing in VLSI[C]//2012 International Symposium on Computer Applications and Industrial Electronics (ISCAIE). IEEE, 2012: 43-47.
- [45] NATH S, GHOSH S, SARKAR S K. A novel approach to discrete particle swarm optimization for efficient routing in VLSI design[C]//2015 4th International Conference on Reliability, Infocom Technologies and Optimization (ICRITO)(Trends and Future Directions). IEEE, 2015: 1-4.
- [46] 洪先龙, 严晓浪, 乔长阁. 超大规模集成电路布图理论与算法 [M]. 科学出版社, 1998.
- [47] 潘松, 黄继业. EDA 技术实用教程: 第 10 卷 [M]. 科学出版社, 2002.
- [48] 李毛毛. 蚁群算法在集成电路布线问题中的应用 [D]. 兰州大学, 2008.
- [49] RIVEST R L, FIDUCCIA C M. A 'greedy' channel router[J]. Computer-Aided Design, 1983, 15(3): 135-140.
- [50] ADYA S N, MARKOV I L. Fixed-outline floorplanning: Enabling hierarchical design[J]. IEEE

- Transactions on Very Large Scale Integration (VLSI) Systems, 2003, 11(6): 1120-1135.
- [51] ALPERT C J, MEHTA D P, SAPATNEKAR S S. Handbook of algorithms for physical design automation[M]. CRC press, 2008.
 - [52] MASSIER T, GRAEB H, SCHLICHTMANN U. The sizing rules method for CMOS and bipolar analog integrated circuit synthesis[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2008, 27(12): 2209-2222.
 - [53] DESSOUKY M, LOUERAT M M. A layout approach for electrical and physical design integration of high-performance analog circuits[C]//Proceedings IEEE 2000 First International Symposium on Quality Electronic Design (Cat. No. PR00525). IEEE, 2000: 291-298.
 - [54] ZHANG Y, HU X, FENG X, et al. An analysis of power dissipation analysis and power dissipation optimization methods in digital chip layout design[C]//2019 IEEE 19th International Conference on Communication Technology (ICCT). IEEE, 2019: 1468-1471.
 - [55] ARORA T, MOSES M E. Ant colony optimization for power efficient routing in manhattan and non-manhattan VLSI architectures[C]//2009 IEEE Swarm Intelligence Symposium. IEEE, 2009: 137-144.
 - [56] MIRONOV S E, ANDREEV L E. Translator of complex hierarchical vlsi objects layout description from a real form to a virtual layout model[C]//2021 XXIV International Conference on Soft Computing and Measurements (SCM). IEEE, 2021: 81-84.
 - [57] SHAPHIR E, PINTER R Y, WIMER S. Efficient cell-based migration of VLSI layout[J]. Optimization and Engineering, 2015, 16(1): 203-223.
 - [58] TANG X, YUAN X. Technology migration techniques for simplified layouts with restrictive design rules[C]//2006 IEEE/ACM International Conference on Computer Aided Design. IEEE, 2006: 655-660.
 - [59] MIRONOV S E, VASILIEV A Y. Controlling the process of coordination of complex layout objects of microelectronic systems in conditions of uncertainty of design rules[C]//2017 IEEE II International Conference on Control in Technical Systems (CTS). IEEE, 2017: 192-195.
 - [60] BHATTACHARYA P, KHAN A, SARKAR S K. A global routing optimization scheme based on ABC algorithm[M]//Advanced Computing, Networking and Informatics-Volume 2: Advanced Computing, Networking and Informatics-Volume 2. Springer, 2014: 189-197.
 - [61] SHERWANI N A. Algorithms for VLSI physical design automation[M]. Springer Science & Business Media, 2012.
 - [62] RICHARDS D. Complexity of single-layer routing[J]. IEEE Transactions on Computers, 1984, 33(03): 286-288.
 - [63] SZYMANSKI T G. Dogleg channel routing is NP-complete[J]. IEEE Transactions on Computer-

- aided Design of Integrated Circuits and Systems, 1985, 4(1): 31-41.
- [64] BLUM C. Ant colony optimization: Introduction and recent trends[J]. Physics of Life Reviews, 2005, 2(4): 353-373.
- [65] 景阳. 基于蚁群算法的热点话题发现研究 [D]. 南京师范大学, 2013.
- [66] 姜文波. 蚁群算法局部最优解决机制的探讨 [J]. 智能计算机与应用, 2014, 4(03): 53-54+59.
- [67] BULLNHEIMER B, HARTL R F, STRAUSS C. An improved ant system algorithm for the vehicle routing problem[J]. Annals of Operations Research, 1999, 89: 319-328.
- [68] JANGRA R, KAIT R. Analysis and comparison among ant system; ant colony system and MAX-MIN ant system with different parameters setting[C]//2017 3rd International Conference on Computational Intelligence & Communication Technology (CICT). IEEE, 2017: 1-4.
- [69] STU TZLE T, HOOS H H. MAX-MIN ant system[J]. Future Generation Computer Systems, 2000, 16(8): 889-914.
- [70] TEIG S L. The X architecture: Not your father's diagonal wiring[C]//Proceedings of the 2002 international workshop on System-level interconnect prediction. Association for Computing Machinery, 2002: 33-37.
- [71] WEI Y, DONG S, HONG X. APWL-Y: An accurate and efficient wirelength estimation technique for hexagon/triangle placement[J]. Integration, 2007, 40(4): 406-419.
- [72] 高艳明. PCB 自动布线器的选择 —— 关于有网格与无网格布线器的思考 [J]. 电子产品世界, 1996(12): 44-45.
- [73] HIGHTOWER D W. A solution to line-routing problems on the continuous plane[C]//Proceedings of the 6th annual Design Automation Conference. Association for Computing Machinery, 1969: 1-24.
- [74] 张良震, 刘红, 史亮, 等. 遗传算法应用于 VLSI 布局的研究 [J]. 电路与系统学报, 1999(03): 47-53.
- [75] 黄训诚. 基于蚁群算法的超大规模集成电路布线研究 [D]. 西安电子科技大学, 2007.
- [76] 盖宇. 基于人工免疫-蚁群混合算法的 VLSI 布线研究 [D]. 哈尔滨工程大学, 2009.
- [77] 邓爱姣, 李强, 张嘉为. 改进的 Prim 启发式算法在 VLSI 布线中的应用 [J]. 沈阳工业大学学报, 2006(05): 557-559+567.
- [78] BIGALKE S, LIENIG J. Avoidance vs. repair: New approaches to increasing electromigration robustness in VLSI routing[J]. Integration, 2020, 75: 189-198.
- [79] RABANAL P, RODRÍGUEZ I. Hybridizing river formation dynamics and ant colony optimization[C]//European Conference on Artificial Life. Springer, 2009: 424-431.
- [80] RABANAL P, RODRÍGUEZ I, RUBIO F. Using river formation dynamics to design heuristic algorithms[C]//International Conference on Unconventional Computation. Springer, 2007: 163-177.

- [81] RABANAL P, RODRÍGUEZ I. Testing restorable systems by using RFD[C]//International Work-Conference on Artificial Neural Networks. Springer, 2009: 351-358.
- [82] DASH S, JOSHI D, TRIVEDI G. CMOS analog circuit optimization via river formation dynamics[C]//2016 26th International Conference Radioelektronika (Radioelektronika). IEEE, 2016: 51-55.
- [83] MEHRJOO S, KHUNJUSH F. Optimal data aggregation tree in wireless sensor networks based on improved river formation dynamics[J]. Computational Intelligence, 2018, 34(3): 802-820.
- [84] REDLARSKI G, PAŁKOWSKI A, DĄBKOWSKI M. Using river formation dynamics algorithm in mobile robot navigation[C]//Solid State Phenomena: volume 198. Trans Tech Publ, 2013: 138-143.
- [85] THE K S, WONG D F, CONG J. A layout modification approach to via minimization[J]. IEEE transactions on Computer-aided Design of Integrated Circuits and Systems, 1991, 10(4): 536-541.



西安电子科技大学
XIDIAN UNIVERSITY

地址：西安市太白南路2号

邮编：710071

网址：www.xidian.edu.cn